

DEVSECOPS PROJECT REPORT

Secure CI/CD for a 3-Tier Application on AWS

Jenkins pipeline with Trivy, SonarQube and Docker, deployed with Docker Compose on an Ubuntu 26.04 EC2 instance

AWS EC2 | Jenkins | Docker | Docker Compose | Trivy | SonarQube | Docker Scout | Flask | MySQL

Goal	Build, scan, analyse and deploy a Flask + MySQL app through an automated pipeline where security checks run before deployment (shift-left).
Outcome	Pipeline build #4 finished SUCCESS. SonarQube Quality Gate: Passed. Image pushed to Docker Hub. App live on port 5000 (score 15/15 stored in MySQL).
Also covered	Every command used, every screenshot, all 14 errors met on the way and their fixes, lessons learned and hardening advice.
Date	September 2026

Table of Contents

Table of Contents	2
1. Project Overview	4
1.1 What the pipeline does	4
1.2 Results at a glance	4
2. DevSecOps in Simple Terms	4
2.1 Why it matters	4
2.2 Which tool covers which area	5
3. Architecture and Pipeline Design	5
3.1 Pipeline stages	6
3.2 Ports used	6
4. Environment and Prerequisites	6
5. Step-by-Step Implementation	7
Step 1: Launch the EC2 instance	7
Step 2: Open the required ports in the Security Group	7
Step 3: Connect and clone the project scripts	7
Step 4: Install Jenkins (with the Ubuntu 26.04 fixes)	8
Step 5: Install Docker, add Jenkins to the docker group, install Compose	10
Step 6: Install Trivy	10
Step 7: Log in to Docker Hub with an access token	11
Step 8: Install Docker Scout	11
Step 9: Run SonarQube in a container	12
Step 10: Install the Jenkins plugins	13
Step 11: Configure the tools (SonarScanner and Docker)	14
Step 12: Add the Docker Hub credential in Jenkins	15
Step 13: Create a SonarQube token and store it in Jenkins	16
Step 14: Create the SonarQube webhook	16
Step 15: Register the SonarQube server in Jenkins	17
Step 16: Create the pipeline job	18
Step 17: Run the build and open the application	19
Step 18: Verify the database (MySQL inside the container)	21
Step 19: Scan the final image with Trivy on the server	23
6. The Jenkinsfile	24
7. Results and Output	25
7.1 Pipeline outcome	25

7.2 SonarQube analysis	25
7.3 What the application shows	29
8. Errors Faced and How They Were Solved	29
Issues still worth fixing (found while writing this report)	33
9. Quick Verification Cheat Sheet	34
10. What I Learned	34
11. Security Hardening and Next Steps	34
12. Interview Questions From This Project	35
13. Conclusion	36
Credits and references	36

1. Project Overview

This project automates the journey of a small three-tier web application (a DevOps mock-exam site written in Flask, backed by MySQL) from source code in GitHub to a running container on AWS. Instead of testing security at the end, the pipeline scans the source, analyses code quality and scans the built image **before** the application is deployed. This practice is called **shift-left security**, and it is the core idea of DevSecOps.

1.1 What the pipeline does

- **Checks out** the application code from GitHub.
- **Scans the file system with Trivy** for known vulnerabilities in dependencies and for misconfigurations (Dockerfile, compose files).
- **Analyses code quality with SonarQube** (bugs, vulnerabilities, security hotspots, code smells) and reports a Quality Gate result.
- **Builds a Docker image** and **pushes it to Docker Hub** using stored Jenkins credentials.
- **Scans the built image with Trivy** and archives the HTML reports as build artifacts.
- **Deploys with Docker Compose** (Flask container + MySQL container) and verifies that the app answers on port 5000.

1.2 Results at a glance

Item	Result
Jenkins pipeline (job: applicationdeploy)	Build #3 failed at Docker login, build #4 finished SUCCESS after the fix
SonarQube Quality Gate	Passed (project devops-exam-app, server-side processing: Success)
Trivy reports archived	trivy-fs-report.html (18.51 KiB) and trivy-image-report.html (219.95 KiB)
Docker Hub image	<dockerhub-user>/devopsexamapp:latest, 446 MB on disk (108 MB content)
Running containers	flask_app (5000), mysql_db (3306, healthy), sonar (9000)
Application test	Exam submitted from the browser: score 15/15 (100%); row stored in MySQL table results
Issues found and solved	14 (Linux packages, repo signing keys, firewall, credentials, container restarts, typos)

2. DevSecOps in Simple Terms

DevOps = Build + Test + Deploy + Operate. **DevSecOps** = Build + **Security** + Test + Deploy + Operate. Think of an aircraft: in plain DevOps, a security problem might only be noticed while the plane is already in the air. In DevSecOps the problem is found and fixed on the ground, before takeoff. Moving the checks earlier in the pipeline is why it is called *shift-left*.

2.1 Why it matters

- **Prevent vulnerabilities** by catching risky dependencies and insecure configuration early.
- **Improve quality**: automated static analysis keeps bugs and code smells visible.

- **Increase efficiency:** automation replaces repeated manual security checks.
- **Reduce cost:** fixing an issue in a pipeline is cheaper than fixing it after a breach.

2.2 Which tool covers which area

Security area	Typical tool	Used in this project?
Code quality and code security (SAST)	SonarQube	Yes
Container image and file-system scanning	Trivy (also Gype)	Yes
Image supply-chain insight	Docker Scout	Installed and verified
Infrastructure-as-Code security	Checkov	No (natural next step)
Dependency security	Snyk	No (Trivy covers pip packages here)
Secrets detection in git history	Gitleaks	No (recommended addition)

3. Architecture and Pipeline Design

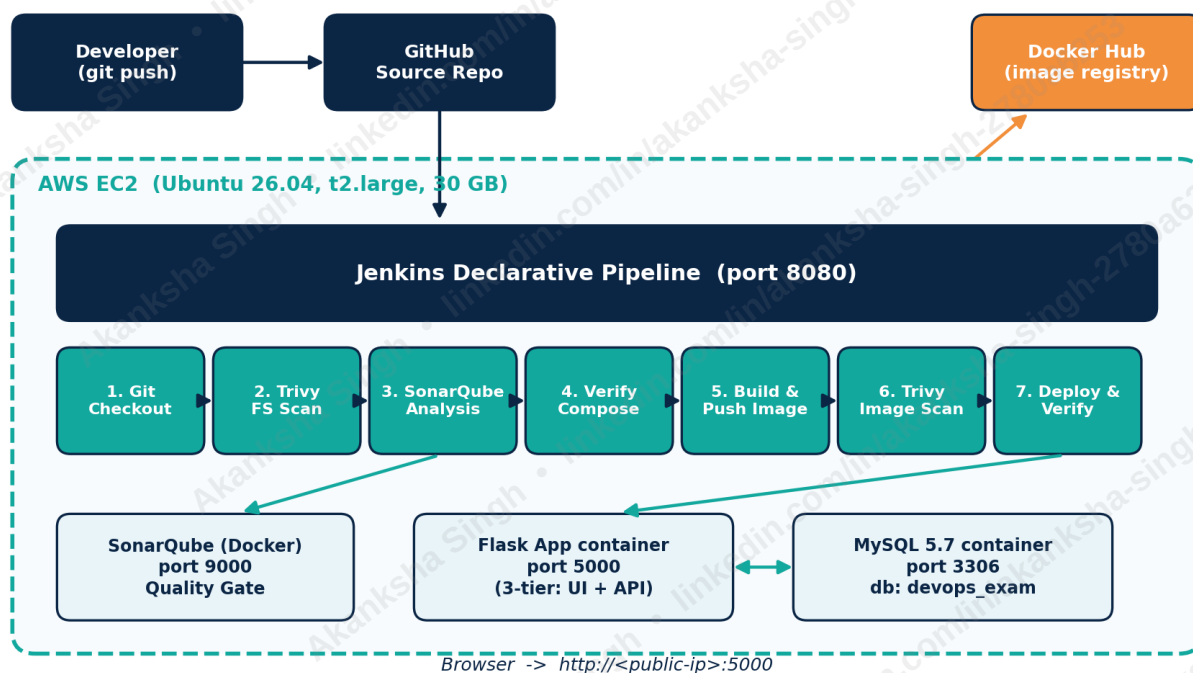


Figure 1. High-level architecture: Jenkins on EC2 orchestrates scanning, build, push and deploy.

3.1 Pipeline stages

#	Stage	What happens	Tool
1	Git Checkout	Clones the master branch of the application repository	git
2	File System Scan	trivy fs scans the workspace, writes trivy-fs-report.html	Trivy
3	SonarQube Analysis	sonar-scanner uploads code; server evaluates the Quality Gate	SonarQube
4	Verify Docker Compose	Fails fast if docker compose is missing	Docker Compose
5	Build and Push Docker Image	Builds image from backend/ and pushes to Docker Hub with stored credentials	Docker
6	Trivy Image Scan	Scans the built image, writes trivy-image-report.html	Trivy
7	Deploy and Verify	docker compose up, waits for MySQL, checks port 5000, prints status and logs	Docker Compose

3.2 Ports used

Port	Service	Purpose
22	SSH	Remote administration (EC2 Instance Connect or MobaXterm)
8080	Jenkins	CI/CD web UI
9000	SonarQube	Code quality dashboard
5000	Flask application	The 3-tier web app
3000, 25, 80, 443	Extra rules from the course template	Opened by the project guide (not required by the pipeline itself)

4. Environment and Prerequisites

Component	Value used
Cloud	AWS EC2 in Asia Pacific (Mumbai), t2.large (2 vCPU, 8 GiB RAM), 30 GB storage, key pair created
Operating system	Ubuntu 26.04 LTS (codename resolute), kernel 7.0.0-aws
Java	OpenJDK 21 JRE headless (Java 17 is no longer in the 26.04 repos)
Jenkins	2.582 (installed from the official apt repository)
SonarQube	sonarqube:lts-community container (server 9.9.8) with SonarScanner CLI 8.1.0.6389
Trivy	Installed from the Aqua Security apt repository
Docker / Compose	Docker Engine, Docker Compose v5.5.1 plugin, Docker Scout CLI v1.24.0
Application	Flask backend + MySQL 5.7 (database devops_exam), source repo forked from CloudDevOpsHub
Accounts needed	AWS account, Docker Hub account (with a Read & Write access token), GitHub account

5. Step-by-Step Implementation

The steps below follow the order in which the project was executed. Commands are shown exactly as run (with the fixes applied), so you can repeat the project from scratch.

Step 1: Launch the EC2 instance

- EC2 console, **Launch instance**. Name: DevOps-app. AMI: Ubuntu. Type: t2.large (2 vCPU, 8 GiB). Create or select a key pair.
- Configure storage: **30 GB** (Jenkins, Docker images and SonarQube need the space).
- Launch and wait for the state to become **Running**.



Figure 2. EC2 instance in the Running state.

Step 2: Open the required ports in the Security Group

Instance, **Security** tab, click the Security Group, **Edit inbound rules**. Add Custom TCP rules for 8080 (Jenkins), 9000 (SonarQube) and 5000 (application), plus the ports below. Each port needs its own rule; a missing rule shows up in the browser as `ERR_CONNECTION_TIMED_OUT` (see Errors 4 and 10).

Type	Protocol	Port range	Source
Custom TCP	TCP	3000	0.0.0.0/0
SSH	TCP	22	0.0.0.0/0
HTTPS	TCP	443	0.0.0.0/0
HTTP	TCP	80	0.0.0.0/0
Custom TCP	TCP	9000	0.0.0.0/0
Custom TCP	TCP	5000	0.0.0.0/0
Custom TCP	TCP	8080	0.0.0.0/0
SMTP	TCP	25	0.0.0.0/0

Figure 3. Inbound rules of the Security Group (3000, 22, 443, 80, 9000, 5000, 8080, 25).

WARNING: The lab uses source 0.0.0.0/0 for convenience. For anything beyond a short-lived lab, restrict 22, 8080 and 9000 to your own IP.

Step 3: Connect and clone the project scripts

Connect with EC2 Instance Connect (browser) or MobaXterm, then clone the repository that contains the numbered installation scripts and make them executable.

```
sudo -i
git clone https://github.com/CloudDevOpsHub/3tierapplicationdeployDevSecOps.git
cd 3tierapplicationdeployDevSecOps
ls
chmod +x *.sh
```

```
ubuntu@ip-172-31-35-232:~$ sudo -i
root@ip-172-31-35-232:~# git clone https://github.com/CloudDevOpsHub/3tierapplicationdeployDevSecOps.git
Cloning into '3tierapplicationdeployDevSecOps'...
remote: Enumerating objects: 55, done.
remote: Counting objects: 100% (23/23), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 55 (delta 21), reused 18 (delta 18), pack-reused 32 (from 1)
Receiving objects: 100% (55/55), 21.35 KiB | 2.67 MiB/s, done.
Resolving deltas: 100% (23/23), done.
root@ip-172-31-35-232:~#
```

Figure 4. Cloning the repository as root.

```
Resolving deltas: 100% (23/23), done.
root@ip-172-31-35-232:~# cd 3tierapplicationdeployDevSecOps
root@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps# ls
1st-Jenkins.sh 3rd-Adduser+python.sh 5th-DockerScout.sh 7th-Pipeline-WithoutTrivy README.md
2nd-Docker.sh 4th-Trivy.sh 6th-RunSonarQube 7th-pipeline-withTrivy
root@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps# chmod +x *.sh3tierapplicationdeployDevSecOps
chmod: cannot access '*.sh3tierapplicationdeployDevSecOps': No such file or directory
root@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps# chmod +x *.sh
root@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps#
```

Figure 5. Listing the scripts. The first chmod failed because of a pasted typo (Error 1); the second one worked.

`chmod +x *.sh` gives execute permission to every file whose name ends in `.sh`.

Step 4: Install Jenkins (with the Ubuntu 26.04 fixes)

The original `1st-Jenkins.sh` targets older Ubuntu releases and failed here twice (Errors 2 and 3): Java 17 is unavailable and the 2023 Jenkins signing key is outdated. The corrected script:

```
#!/bin/bash
set -e
# Install a supported JRE (Java 21)
sudo apt-get update -y
sudo apt-get install -y openjdk-21-jre-headless
# Keyrings folder
sudo mkdir -p /usr/share/keyrings
# Download the CURRENT Jenkins signing key
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian/jenkins.io-2026.key
# Add Jenkins repository
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
# Install Jenkins
sudo apt-get update -y
sudo apt-get install -y jenkins
```

Before re-running after a failed attempt, remove the bad key and repo entry, then run the script and check the service:

```
sudo rm -f /usr/share/keyrings/jenkins-keyring.asc /etc/apt/sources.list.d/jenkins.list
./1st-Jenkins.sh
sudo systemctl status jenkins --no-pager
jenkins --version
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

```

# Update
sudo apt-get update -y

# Install Jenkins
sudo apt-get install jenkins -y
Package openjdk-17-jre-headless is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

Error: Package 'openjdk-17-jre-headless' has no installation candidate
--2026-09-19 21:06:26-- https://pkg.jenkins.io/debian/jenkins.io-2023.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 151.101.210.133, 2a04:4e42:31::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)[151.101.210.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3175 (3.1K) [application/octet-stream]
Saving to: '/usr/share/keyrings/jenkins-keyring.asc'

/usr/share/keyrings/jenkins-key 100%[=====>] 3.10K --.-KB/s in 0s

2026-09-19 21:06:27 (47.1 MB/s) - '/usr/share/keyrings/jenkins-keyring.asc' saved [3175/3175]
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu/resolute InRelease

```

Figure 6. Original script failing: no installation candidate for openjdk-17-jre-headless (Error 2).

```

no VM guests are running extended hypervisor (qemu) binaries on this host.
root@ip-172-31-232:~/3tierapplicationdeployDevSecOps# sudo rm -f /usr/share/keyrings/jenkins-keyring.asc /etc/apt/sources.list.d/jenkins.list
root@ip-172-31-232:~/3tierapplicationdeployDevSecOps# #!/bin/bash
set -e

# Install a supported JRE (adjust version to what apt-cache shows)
sudo apt-get update -y
sudo apt-get install -y openjdk-21-jre-headless

# Keyrings folder
sudo mkdir -p /usr/share/keyrings

# Download the CURRENT Jenkins signing key
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian/jenkins.io-2026.key

# Add Jenkins repo
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# Install Jenkins
sudo apt-get update -y
sudo apt-get install -y jenkins
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu/resolute InRelease

```

Figure 7. Corrected script running (Java 21 and the 2026 signing key).

Open `http://<public-ip>:8080`, paste the initial admin password, install the suggested plugins and create the admin user.

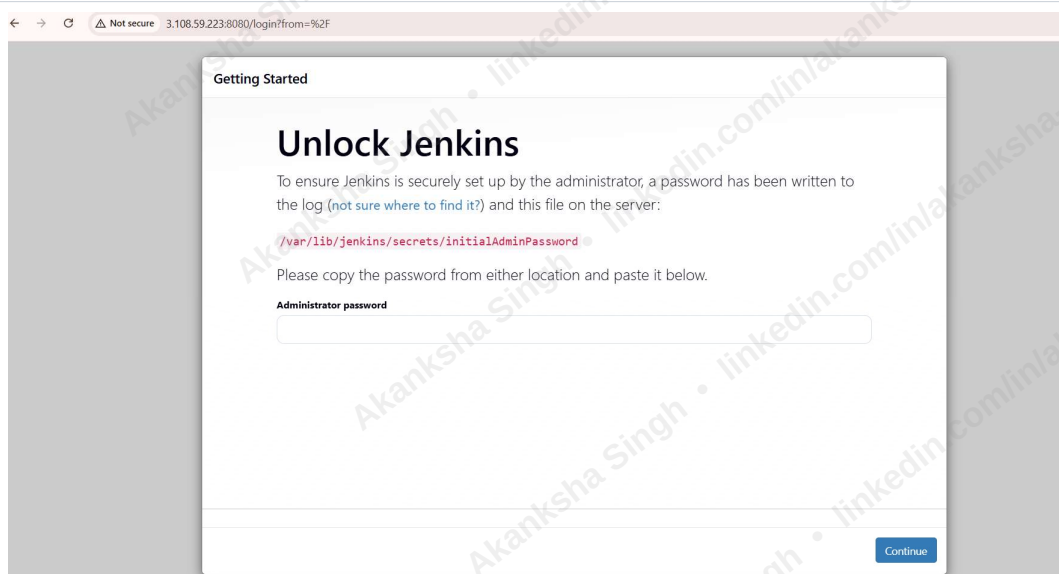


Figure 8. Unlock Jenkins screen on port 8080.

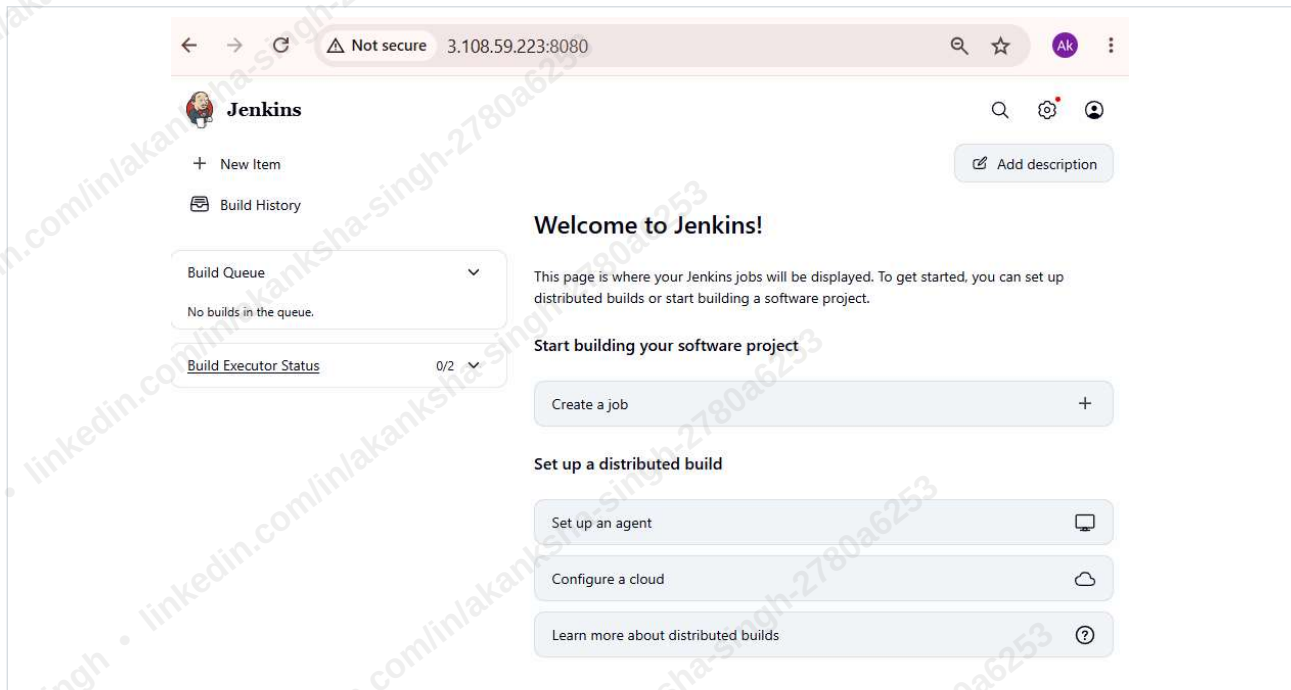


Figure 9. Jenkins dashboard after setup.

Step 5: Install Docker, add Jenkins to the docker group, install Compose

Run `./2nd-Docker.sh` to install Docker Engine, then `./3rd-Adduser+python.sh`. The second script is equivalent to these commands:

```
./2nd-Docker.sh
# 3rd-Adduser+python.sh does the following:
sudo usermod -aG docker jenkins      # let Jenkins use Docker
sudo systemctl restart jenkins       # apply the group change
sudo -u jenkins docker ps            # verify Jenkins can reach Docker
sudo apt-get update
sudo apt-get install -y python3-venv python3-pip
sudo apt-get install -y docker-compose-plugin
```

```
no images are running
root@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps# ./3rd-Adduser+python.sh
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 https://download.docker.com/linux/ubuntu resolute InRelease
```

Figure 10. Running `3rd-Adduser+python.sh`: `docker ps` works for the jenkins user.

Step 6: Install Trivy

```
./4th-Trivy.sh
# Equivalent commands inside the script:
sudo apt-get install -y wget apt-transport-https gnupg
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | gpg --dearmor | \
  sudo tee /usr/share/keyrings/trivy.gpg > /dev/null
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] \
  https://aquasecurity.github.io/trivy-repo/deb generic main" | \
  sudo tee -a /etc/apt/sources.list.d/trivy.list
sudo apt-get update && sudo apt-get install -y trivy
trivy --version
```

```
0 upgraded, 0 newly installed, 0 to remove and 163 not upgraded.
root@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps# ./4th-Trivy.sh
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following additional packages will be installed:
```

Figure 11. Running 4th-Trivy.sh.

Step 7: Log in to Docker Hub with an access token

Create a token at Docker Hub, **Account settings, Personal access tokens, Generate new token** with **Read & Write** permission. Copy it immediately; Docker shows it only once. Use the token instead of your password.

```
docker login -u <your-dockerhub-username>
# Password: paste the access token (nothing is displayed while pasting)
```

```
Sep 19 21:34:49 ip-172-31-35-232 jenkins[10518]: 2020-09-19 21:34:49.0000 [id-30]
Sep 19 21:34:49 ip-172-31-35-232 systemd[1]: Started jenkins.service - Jenkins Continuous I
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker login -u 22bcc70096

Info → A Personal Access Token (PAT) can be used instead.
       To create a PAT, visit https://app.docker.com/settings

Password: █
```

Figure 12. docker login prompting for the password.

```
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker login -u 22bcc70096

Info → A Personal Access Token (PAT) can be used instead.
       To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ █
```

Figure 13. Login Succeeded.

IMPORTANT: Never publish or commit your token. If a token is ever shown on screen or in a document, revoke it in Docker Hub and create a new one.

Step 8: Install Docker Scout

```
curl -sSfL \
  https://raw.githubusercontent.com/docker/scout-cli/main/install.sh | \
  sudo sh -s -- -b /usr/local/bin
docker scout version
```

```
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ curl -sSfL https://raw.githubusercontent.com/docker/scout-cli/main/
install.sh | sh -s -- -b /usr/local/bin
sudo chmod 777 /var/run/docker.sock
[info] fetching release script for tag='v1.24.0'
[info] using release tag='v1.24.0' version='1.24.0' os='linux' arch='amd64'
install: Permission denied
[error] failed to install docker-scout
```

Figure 14. Failure when sudo was placed on the wrong side of the pipe (Error 6).

```
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ curl -sSfL https://raw.githubusercontent.com/docker/scout-cli/main/install.sh | sudo sh -s -- -b /usr/local/bin
[info] fetching release script for tag='v1.24.0'
[info] using release tag='v1.24.0' version='1.24.0' os='linux' arch='amd64'
[info] installed /usr/local/bin/docker-scout
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$
```

Figure 15. Successful installation of docker-scout.

Step 9: Run SonarQube in a container

```
docker run -d --name sonar --restart unless-stopped -p 9000:9000 sonarqube:lts-community
docker images
docker ps
```

Open <http://<public-ip>:9000>. Default login is **admin / admin**; SonarQube asks you to set a new password on first login.

NOTE: The project originally ran the container without `--restart`. After an EC2 reboot the containers stayed stopped (Error 10), so the flag is added here.

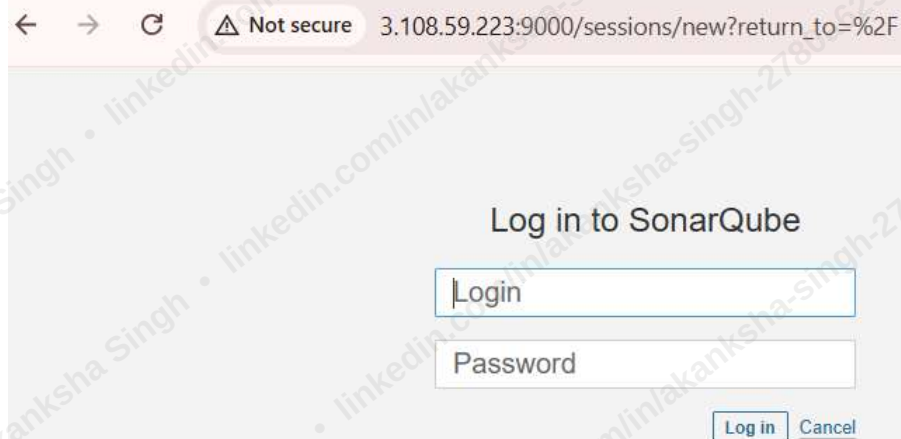


Figure 16. SonarQube login page on port 9000.

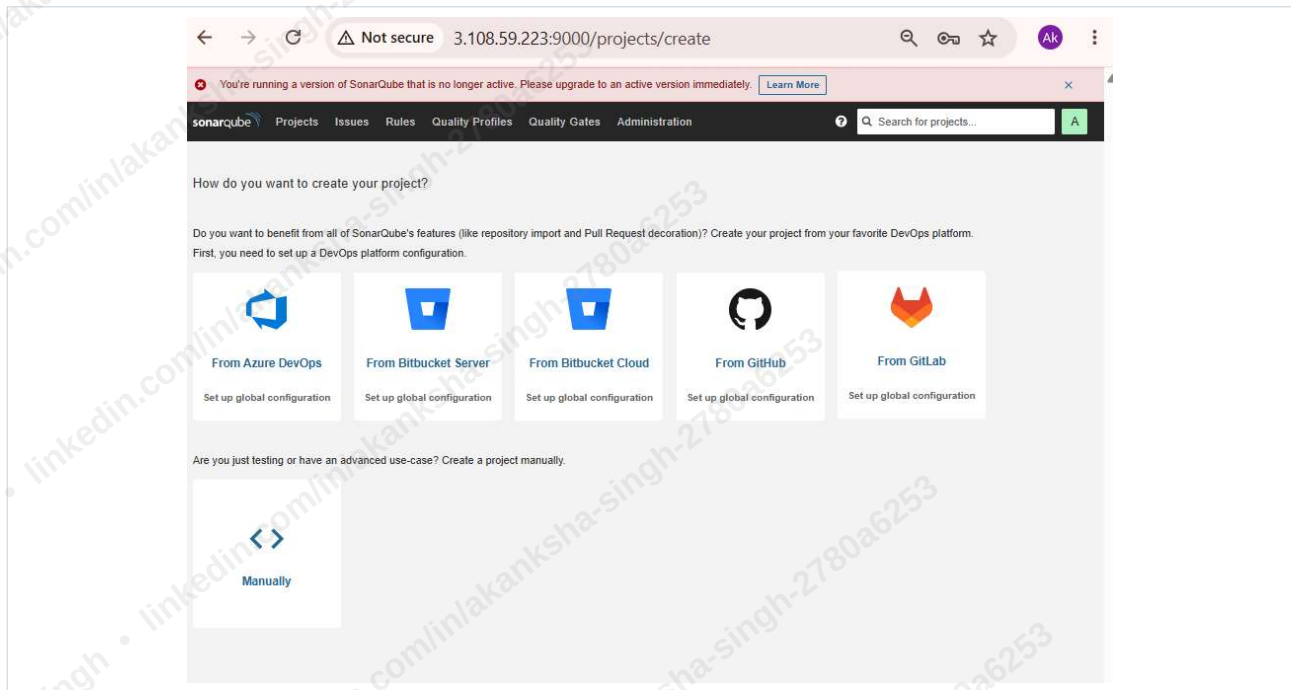


Figure 17. SonarQube after login (project creation page).

Step 10: Install the Jenkins plugins

Manage Jenkins, Plugins, Available plugins. Install: SonarQube Scanner, Docker, Docker Commons, Docker Pipeline, Docker API, docker-build-step and Pipeline Stage View. Restart Jenkins afterwards.

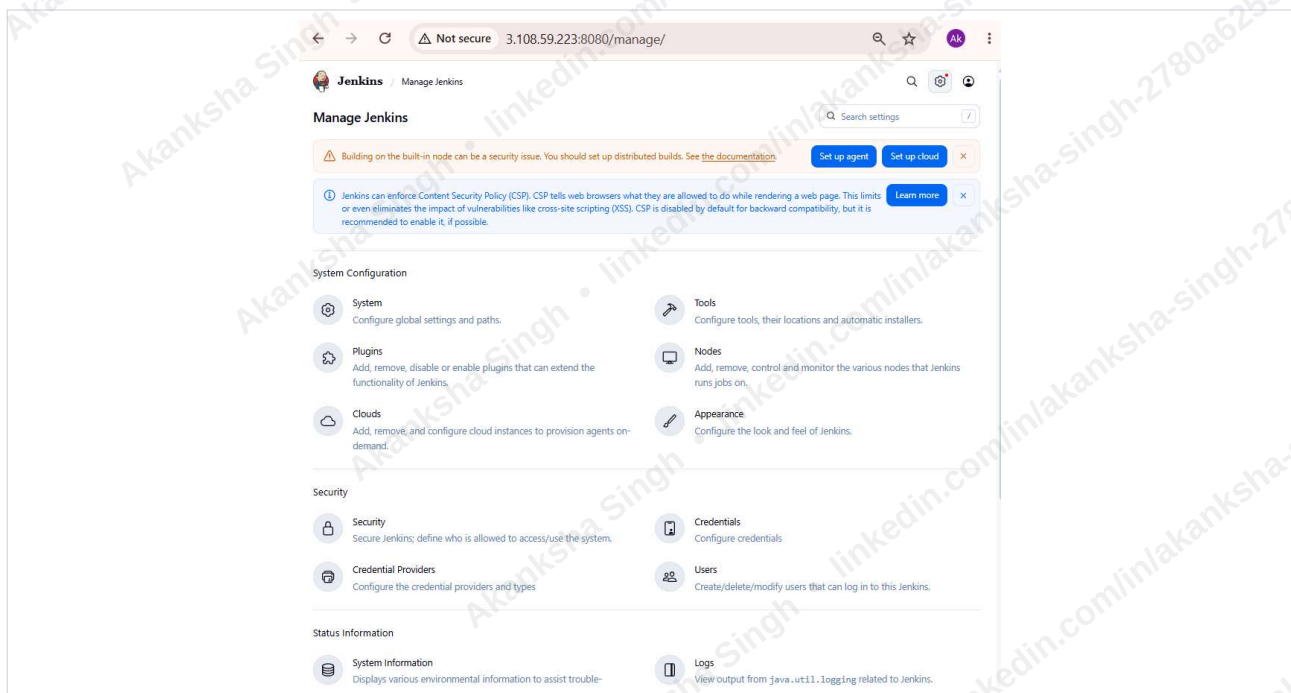


Figure 18. Manage Jenkins page.

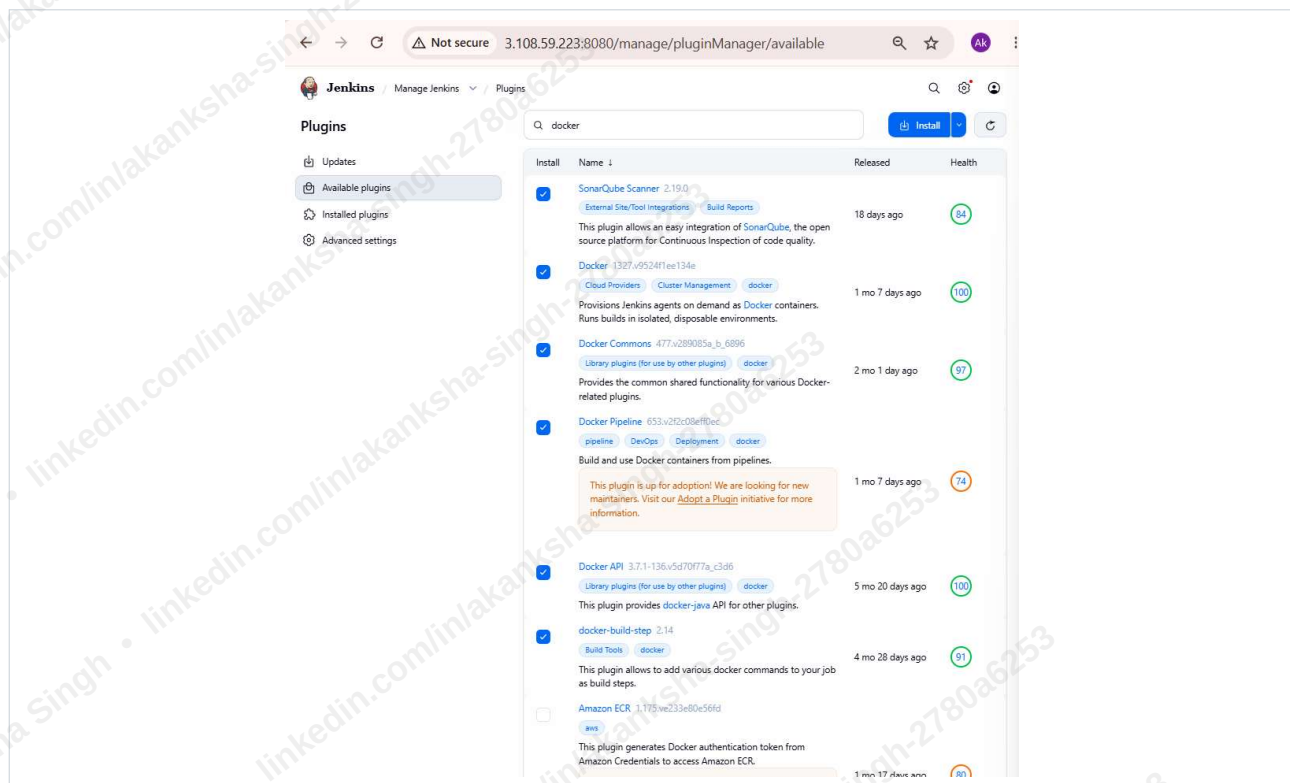


Figure 19. Selecting the Docker and SonarQube plugins.

Step 11: Configure the tools (SonarScanner and Docker)

Manage Jenkins, Tools. Add a SonarQube Scanner named sonar-scanner (install automatically from Maven Central, version 8.1.0.6389) and a Docker installation named docker (install automatically, download from docker.com, latest). The name sonar-scanner is referenced by the Jenkinsfile.

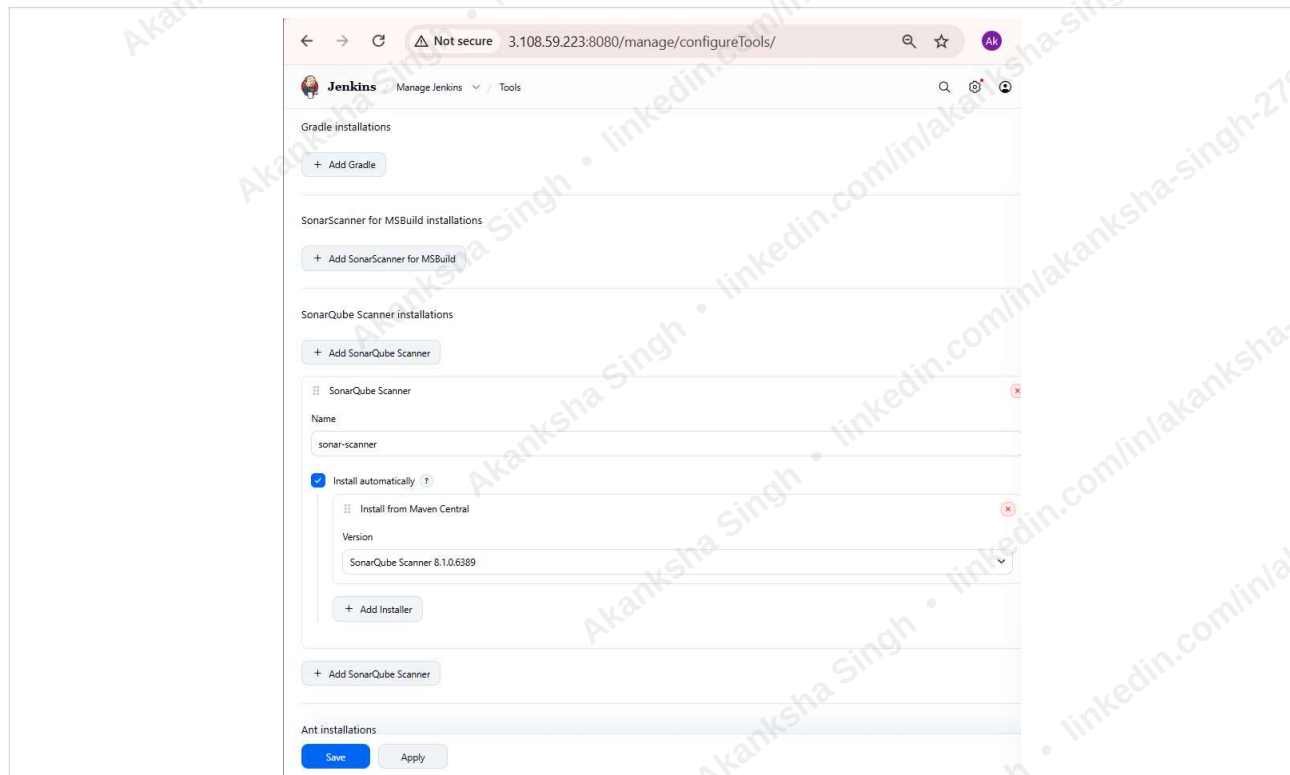
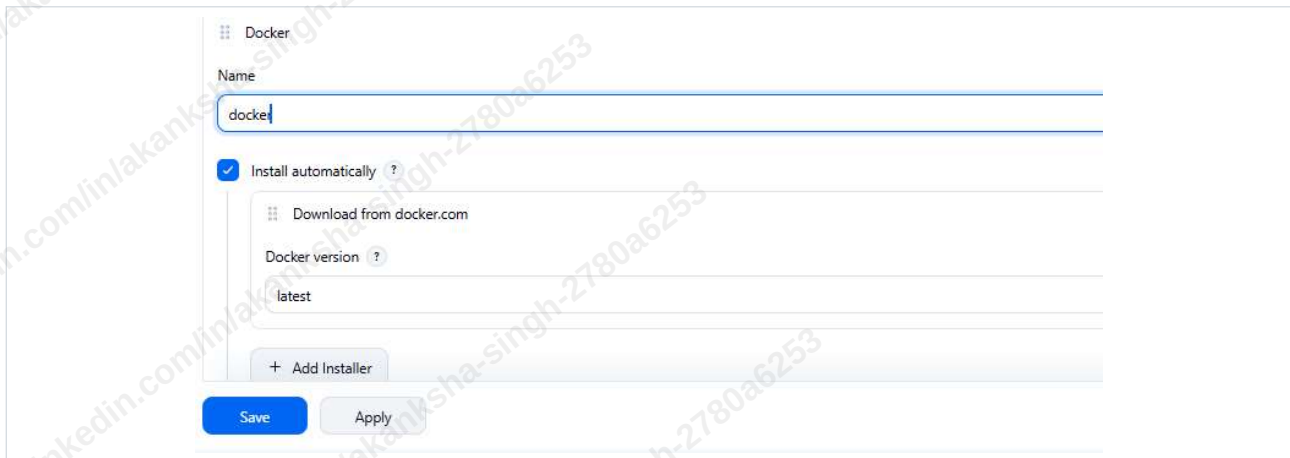


Figure 20. SonarQube Scanner configured as sonar-scanner.

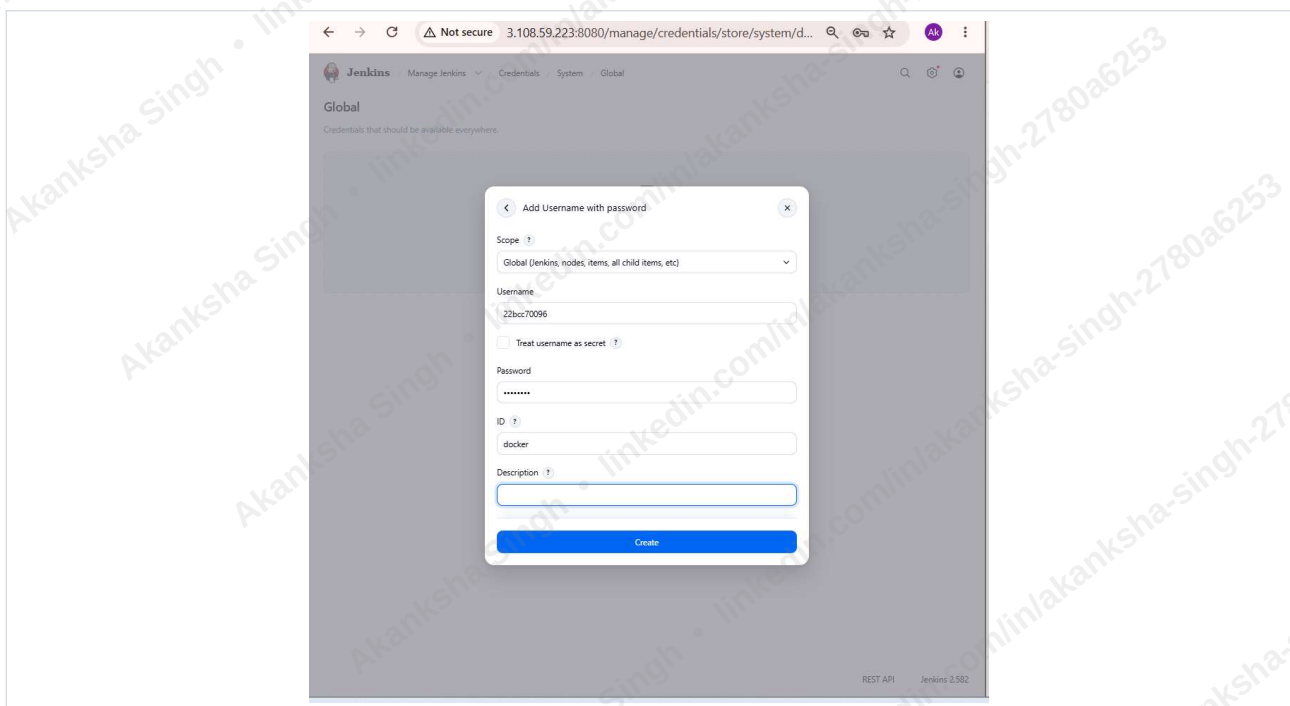


The screenshot shows the Jenkins 'Docker' tool configuration page. The 'Name' field is set to 'docker'. The 'Install automatically' checkbox is checked. Under 'Download from docker.com', the 'Docker version' is set to 'latest'. There are 'Save' and 'Apply' buttons at the bottom.

Figure 21. Docker tool configuration.

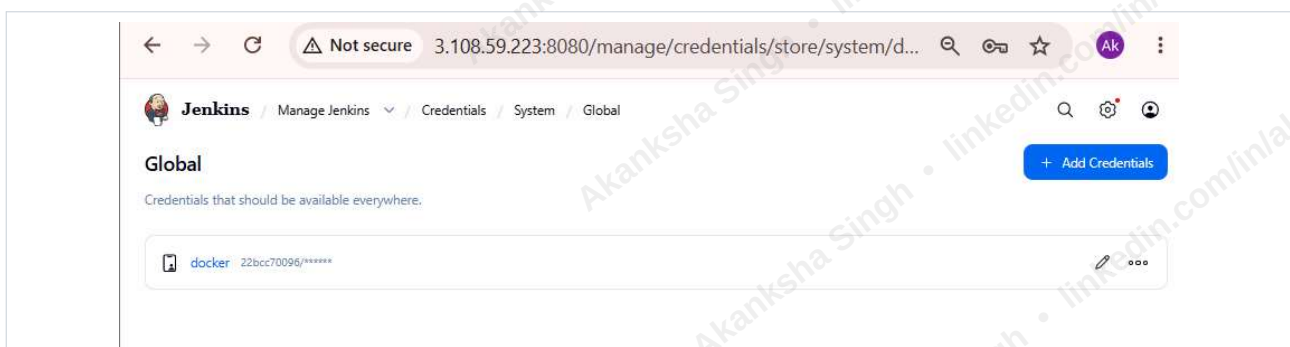
Step 12: Add the Docker Hub credential in Jenkins

Manage Jenkins, Credentials, System, Global credentials, Add Credentials: Kind *Username with password*, Scope *Global*, Username = your Docker Hub username, Password = the **access token**, ID = **docker**. The ID must match `credentialsId: 'docker'` in the `Jenkinsfile`.



The screenshot shows the Jenkins 'Global' credentials page with a modal dialog open to 'Add Username with password'. The 'Scope' is set to 'Global (Jenkins, nodes, items, all child items, etc)'. The 'Username' is '22bcc70096'. The 'ID' is 'docker'. The 'Password' field is masked with asterisks. There is a 'Create' button at the bottom.

Figure 22. Adding the docker credential (password field hidden).



The screenshot shows the Jenkins 'Global' credentials page. A credential named 'docker' with ID '22bcc70096' is listed. There is an 'Add Credentials' button at the top right.

Figure 23. Credential saved.

Step 13: Create a SonarQube token and store it in Jenkins

- SonarQube: **Administration, Security, Users**, open the Tokens icon, generate a token named *token* and copy it (it is shown only once).
- Jenkins: add a new credential of kind **Secret text**, paste the token, ID = sonar-token.

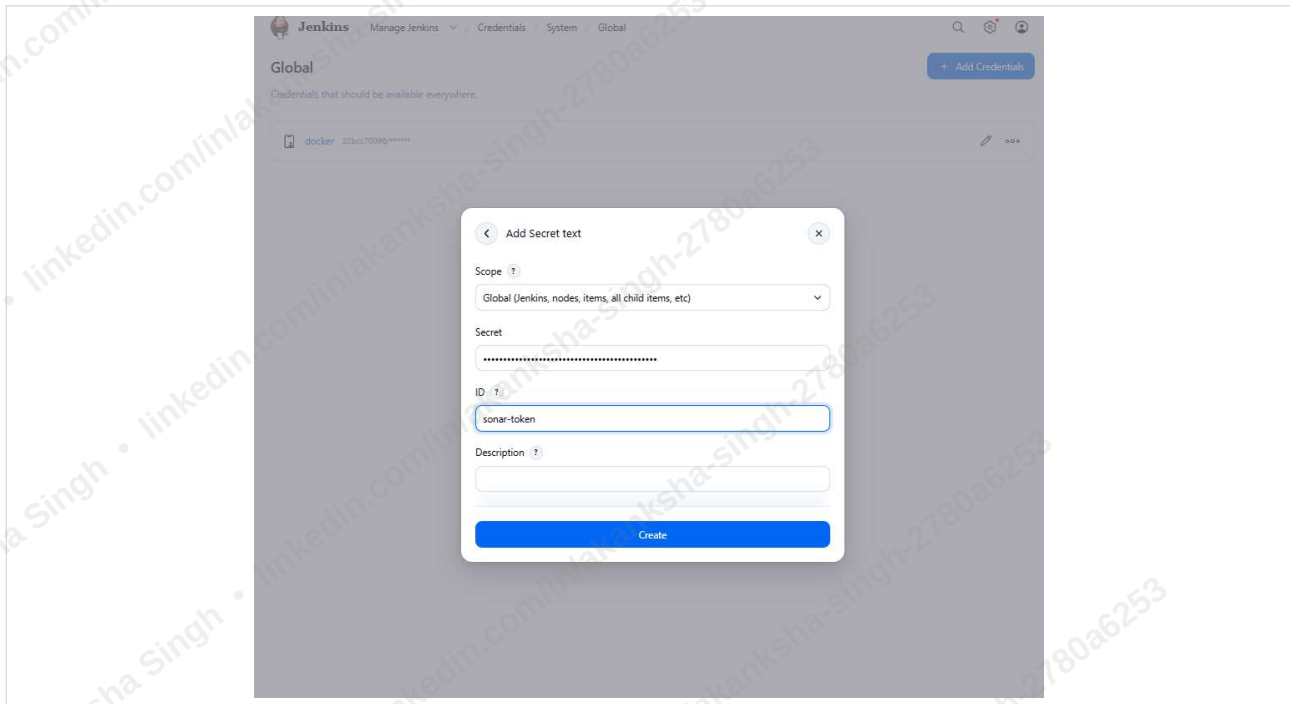


Figure 24. Adding the SonarQube token as Secret text with ID sonar-token.

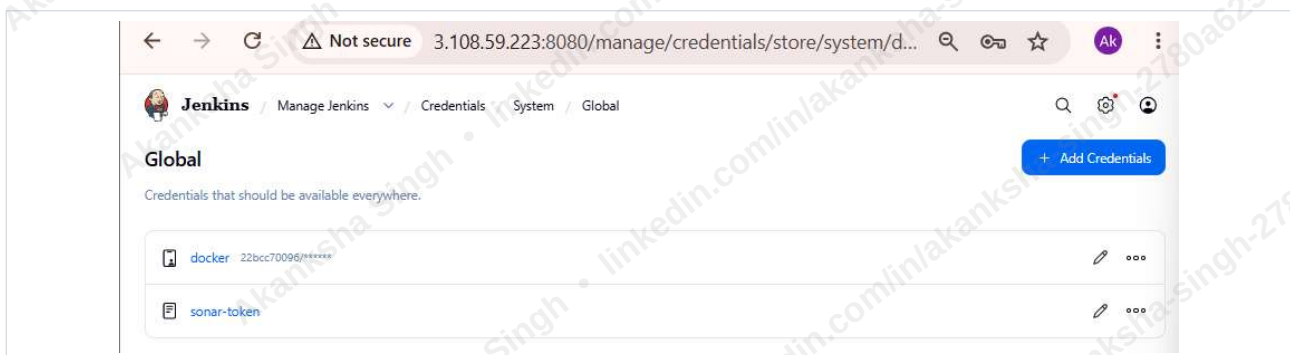


Figure 25. Both credentials present: docker and sonar-token.

Step 14: Create the SonarQube webhook

The webhook lets SonarQube notify Jenkins when analysis finishes, so Jenkins can read the Quality Gate. **Administration, Configuration, Webhooks, Create**: name *jenkins*, URL `http://<public-ip>:8080/sonarqube-webhook/`.

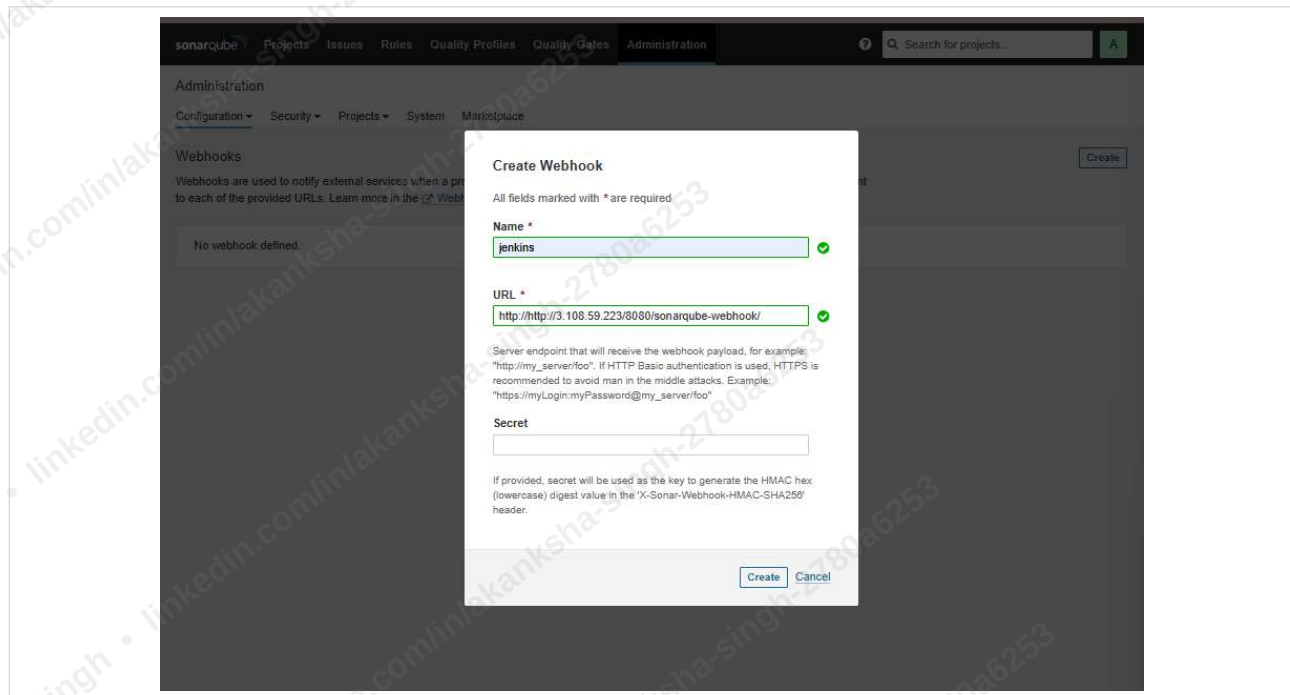


Figure 26. Creating the webhook.

WARNING: In the screenshot the URL was typed as `http://http://...` (the scheme is doubled). It should start with a single `http://`. See "Issues still worth fixing" in Section 8.

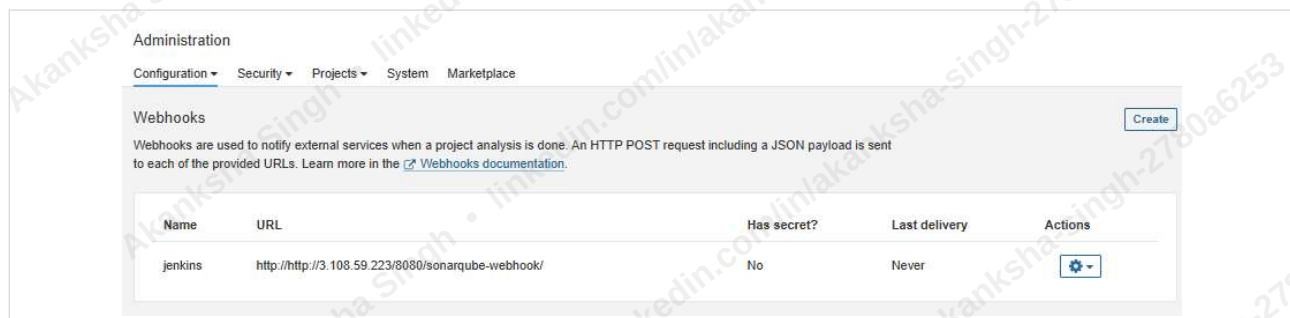


Figure 27. Webhook listed in SonarQube.

Step 15: Register the SonarQube server in Jenkins

Manage Jenkins, System, SonarQube servers, Add SonarQube: Name sonar (referenced by `withSonarQubeEnv('sonar')`), Server URL `http://<public-ip>:9000` with **no trailing slash**, and authentication token sonar-token (select it from the dropdown, then Apply and Save).

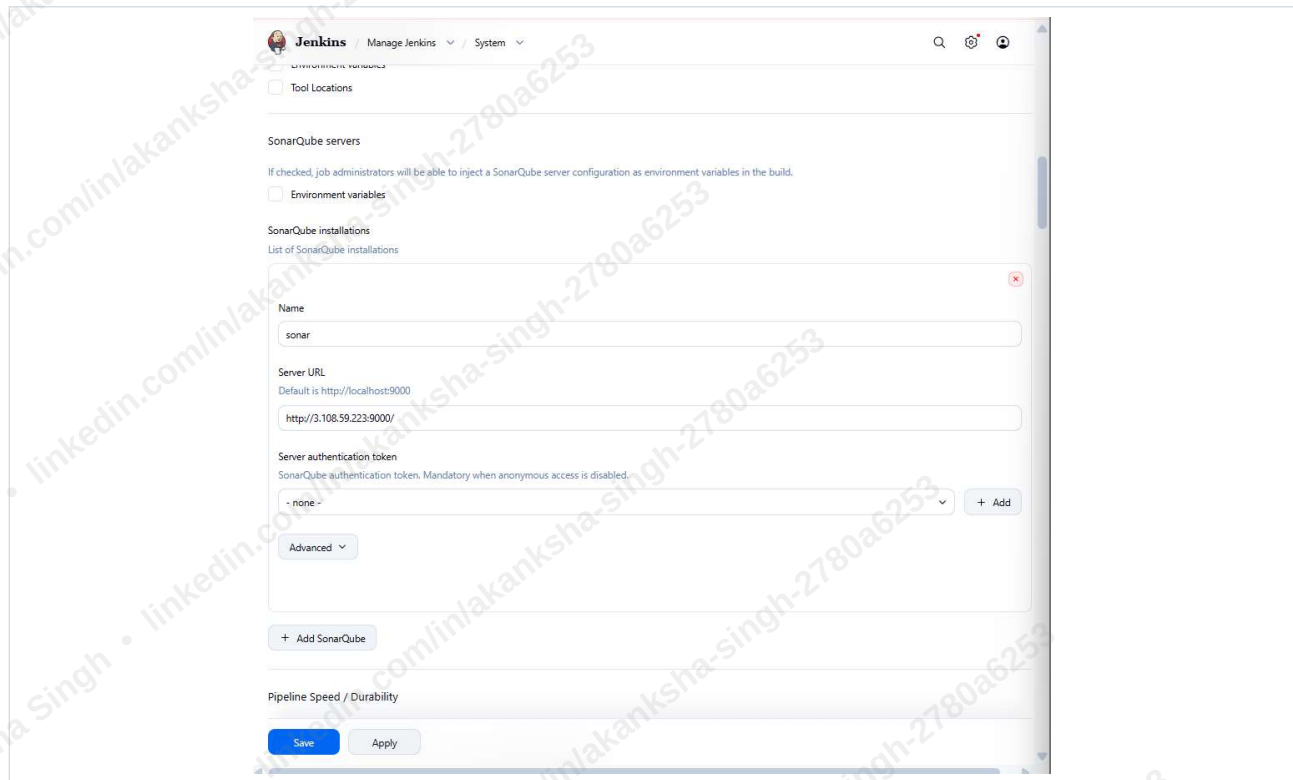


Figure 28. Jenkins SonarQube server section. Jenkins adds a trailing / automatically; remove it and select sonar-token.

Step 16: Create the pipeline job

Dashboard, **New Item**, name `applicationdeploy`, type **Pipeline**. In Configure, set Definition to `Pipeline script` and paste the Jenkinsfile from Section 6.

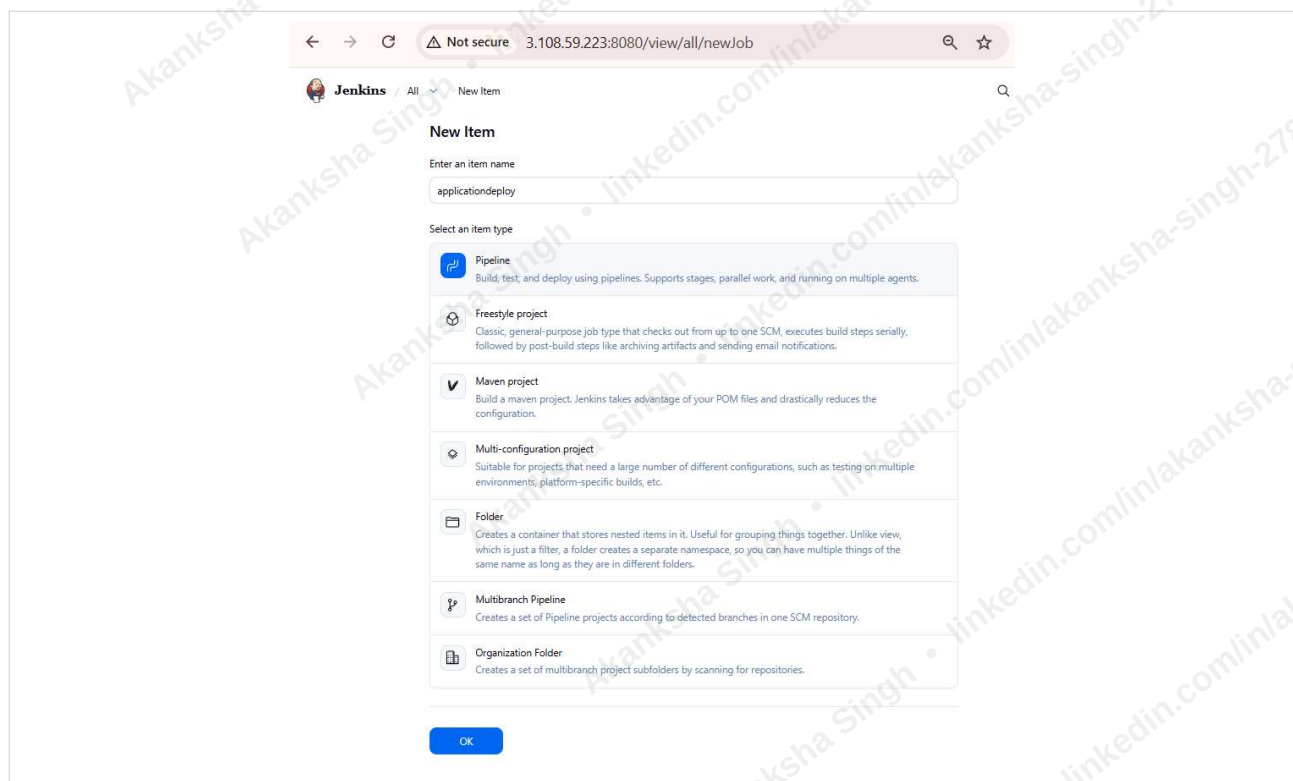


Figure 29. Creating the applicationdeploy pipeline item.

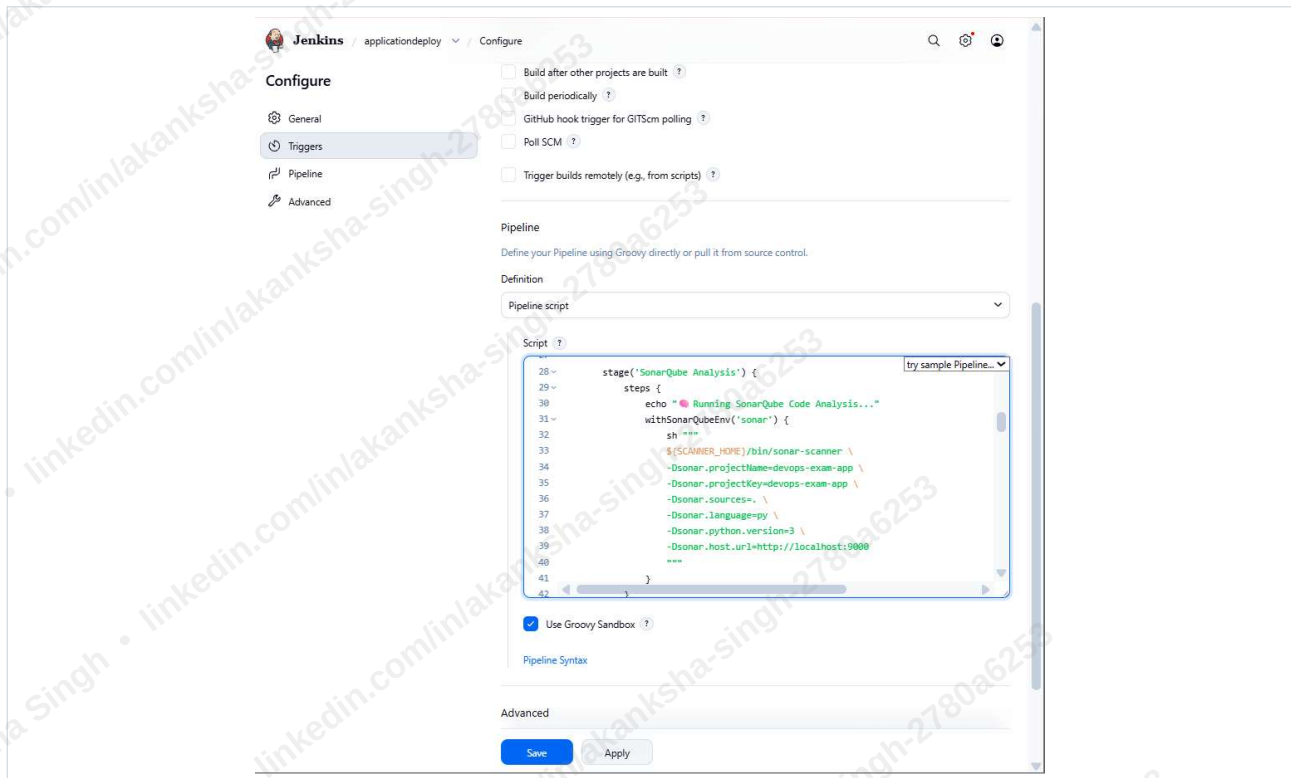


Figure 30. Pipeline script pasted into the job configuration.

Step 17: Run the build and open the application

Click **Build Now** and watch **Console Output**. Build #3 failed at the Docker login (Error 9). After recreating the credential with the access token and correcting the image name, build #4 passed.

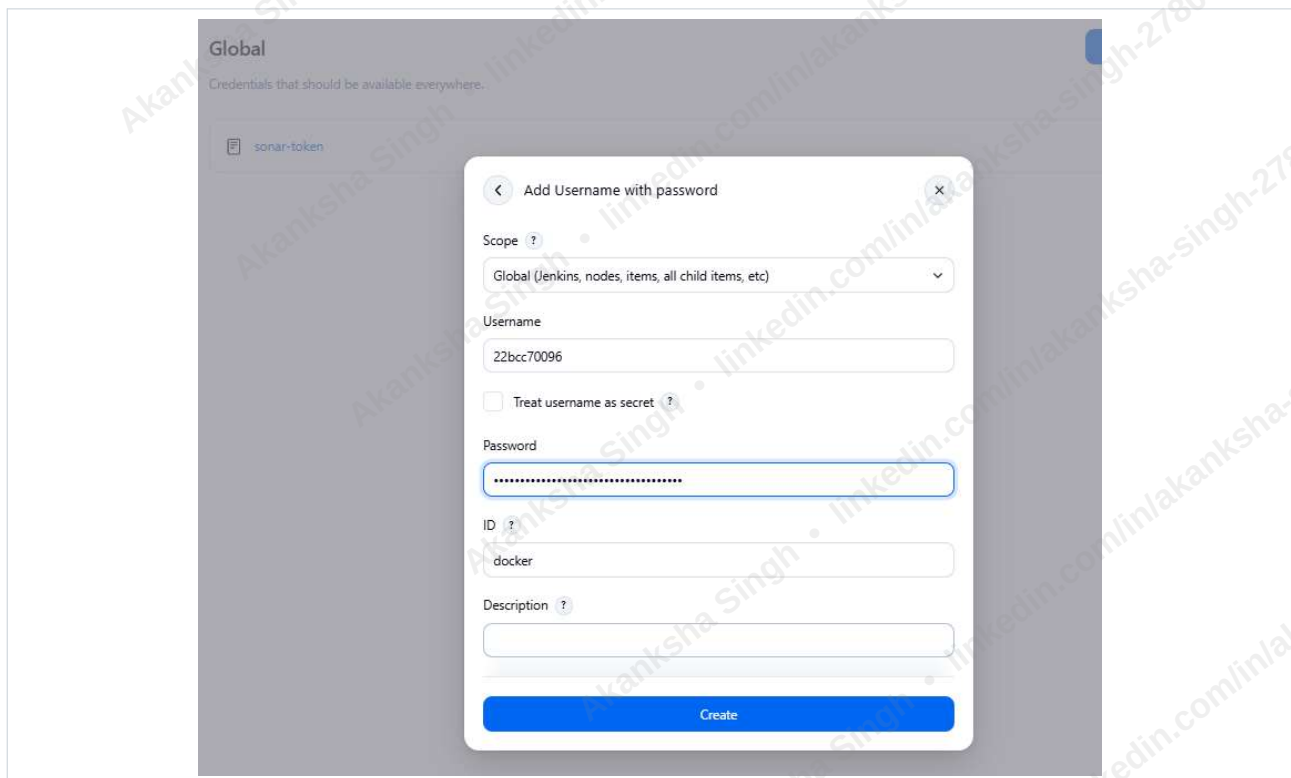


Figure 31. Recreating the docker credential with the access token as the password.

Jenkins / applicationdeploy

Status

applicationdeploy

Last Successful Artifacts

- trivy-fs-report.html 18.51 KiB view
- trivy-image-report.html 219.95 KiB view

SonarQube Quality Gate

devops-exam-app **Passed**

server-side processing: **Success**

Permalinks

- Last build (#4), 2 min 14 sec ago
- Last stable build (#4), 2 min 14 sec ago
- Last successful build (#4), 2 min 14 sec ago
- Last failed build (#3), 6 min 3 sec ago
- Last unsuccessful build (#3), 6 min 3 sec ago
- Last completed build (#4), 2 min 14 sec ago

Builds

Filter

Today

#4 11:02 PM

Figure 32. Job page: build #4 successful, Trivy reports archived, SonarQube Quality Gate Passed.

```

+ flask_app | * environment: production
flask_app | WARNING: This is a development server. Do not use it in a production
deployment.
flask_app | Use a production WSGI server instead.
flask_app | * Debug mode: off
flask_app | * Running on all addresses.
flask_app | WARNING: This is a development server. Do not use it in a production
deployment.
flask_app | * Running on http://172.18.0.3:5000/ (Press CTRL+C to quit)
flask_app | 172.18.0.1 - - [19/Sep/2026 23:04:06] "HEAD / HTTP/1.1" 200 -
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] echo
Deployment successful!
[Pipeline] sh
+ docker compose ps
time="2026-09-19T23:04:06Z" level=warning
msg="/var/lib/jenkins/workspace/applicationdeploy/docker-compose.yml: the attribute
'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME          IMAGE          COMMAND                  SERVICE    CREATED
STATUS        PORTS
flask_app     applicationdeploy-flask    "python app.py"         flask      23 seconds ago
Up 12 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
mysql_db      mysql:5.7        "docker-entrypoint.s..." mysql      26 seconds ago
Up 23 seconds (healthy)  0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Figure 33. End of the console output: Deployment successful, both containers Up, Finished: SUCCESS.

Finally open `http://<public-ip>:5000` (note the **colon** before 5000).

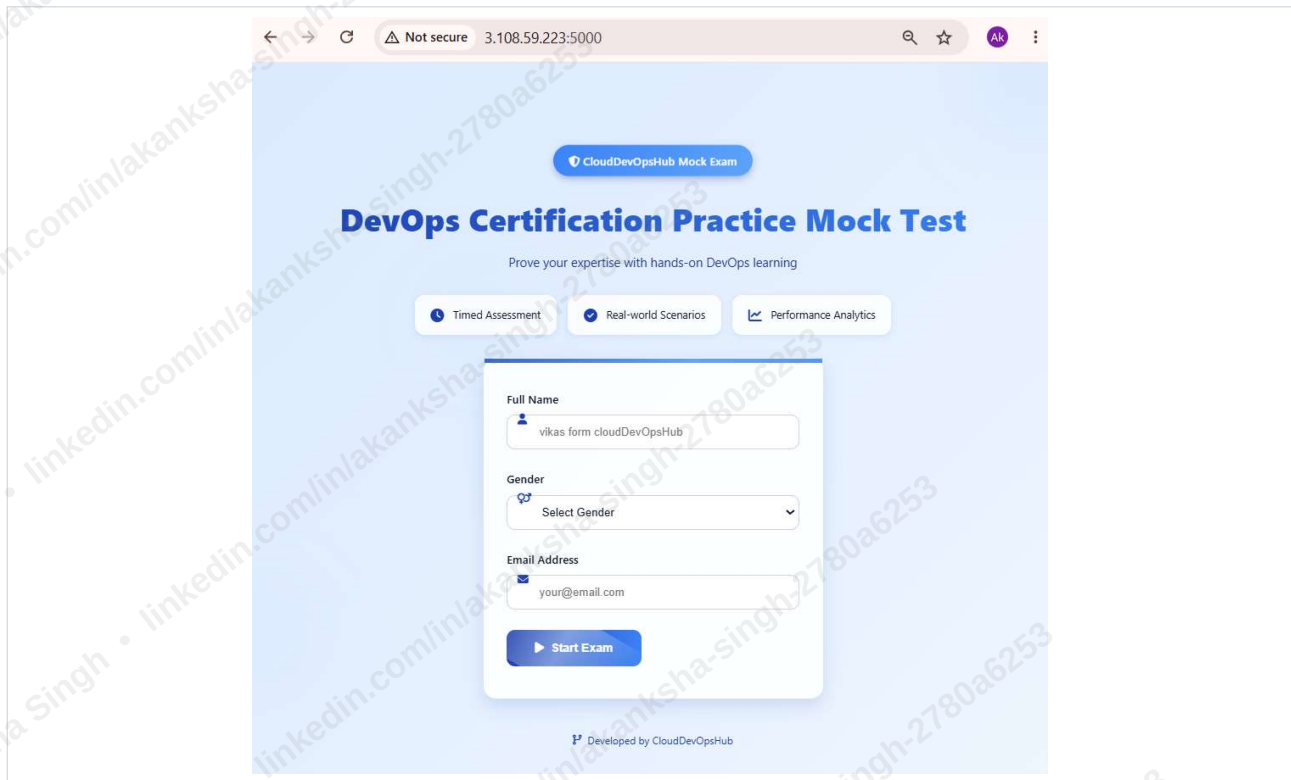


Figure 34. The deployed application: DevOps Certification Practice Mock Test.

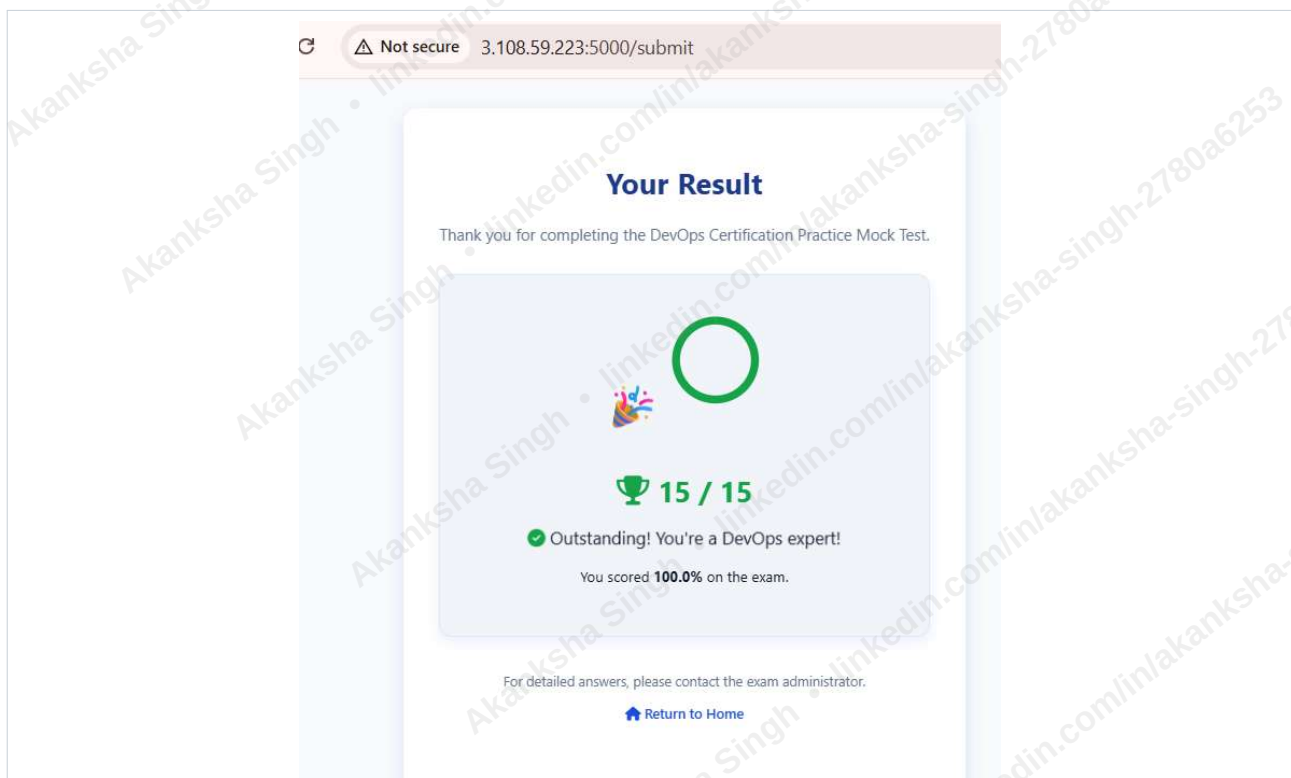


Figure 35. Result page after submitting the exam: 15/15 (100%).

Step 18: Verify the database (MySQL inside the container)

The database runs inside the `mysql_db` container, so connect through Docker rather than a local mysql client. Password: `rootpass` (set in the compose file for this lab).

```
docker exec -it mysql_db mysql -u root -p
SHOW DATABASES;
```

```
USE devops_exam;
SHOW TABLES;
DESCRIBE results;
SELECT * FROM results;
SELECT * FROM results ORDER BY score DESC;
SELECT AVG(score) FROM results;
SELECT COUNT(*) FROM results;    -- no space between COUNT and (*)
```

```
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker exec -it mysql_db mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 850
Server version: 5.7.44 MySQL Community Server (GPL)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Figure 36. Connecting to MySQL inside the container.

```
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| devops_exam |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql> use devops_exam;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_devops_exam |
+-----+
| results |
+-----+
1 row in set (0.00 sec)

mysql> describe results;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| username | varchar(255) | NO | | NULL | |
| gender | varchar(50) | YES | | NULL | |
| email | varchar(255) | YES | | NULL | |
| score | int(11) | YES | | NULL | |
| created_at | timestamp | NO | | CURRENT_TIMESTAMP | |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> select * from results;
+-----+-----+-----+-----+-----+-----+
| id | username | gender | email | score | created_at |
+-----+-----+-----+-----+-----+-----+
| 1 | Ak singh | Female | | 15 | 2026-09-20 08:15:44 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Figure 37. Tables, structure of results and stored record (email redacted).

```
mysql> select * from results ORDER BY score DESC;
+----+-----+-----+-----+-----+-----+
| id | username | gender | email | score | created_at |
+----+-----+-----+-----+-----+-----+
| 1 | Ak singh | Female | | 15 | 2026-09-20 08:15:44 |
+----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> select avg(score) from results;
+-----+
| avg(score) |
+-----+
| 15.0000 |
+-----+
1 row in set (0.00 sec)

mysql> select count (*) from results;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '*) from results' at line 1
mysql> select count(*) from results;
+-----+
| count(*) |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 38. Sorted results, average score and count (email redacted). The first COUNT (*) shows the syntax error from Error 13.

Step 19: Scan the final image with Trivy on the server

```
docker images
trivy image <dockerhub-user>/devopsexamapp:latest
trivy image --severity HIGH,CRITICAL <dockerhub-user>/devopsexamapp:latest
```

```
mysql> select count(*) from results;
+-----+
| count(*) |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)

mysql> exit
Bye
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
0464cce93a88   applicationdeploy-flask             "python app.py"         10 hours ago   Up 2 hours   0.0.0.0:5000->5000/tcp
, [::]:5000->5000/tcp
38b18c1a67f0   mysql:5.7                          "docker-entrypoint.s..." 10 hours ago   Up 2 hours (healthy) 0.0.0.0:3306->3306/tcp
, [::]:3306->3306/tcp, 33060/tcp
e03a0a889e7f   sonarqube:lts-community            "/opt/sonarqube/dock..." 12 hours ago   Up 2 hours   0.0.0.0:9000->9000/tcp
, [::]:9000->9000/tcp
sonar
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ cd
ubuntu@ip-172-31-35-232:~$ cd
ubuntu@ip-172-31-35-232:~$ cd..
cd..: command not found
ubuntu@ip-172-31-35-232:~$ cd ..
ubuntu@ip-172-31-35-232:/home$ cd
ubuntu@ip-172-31-35-232:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
0464cce93a88   applicationdeploy-flask             "python app.py"         10 hours ago   Up 2 hours   0.0.0.0:5000->5000/tcp
, [::]:5000->5000/tcp
38b18c1a67f0   mysql:5.7                          "docker-entrypoint.s..." 10 hours ago   Up 2 hours (healthy) 0.0.0.0:3306->3306/tcp
, [::]:3306->3306/tcp, 33060/tcp
e03a0a889e7f   sonarqube:lts-community            "/opt/sonarqube/dock..." 12 hours ago   Up 2 hours   0.0.0.0:9000->9000/tcp
, [::]:9000->9000/tcp
sonar
ubuntu@ip-172-31-35-232:~$ docker images
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
22bcc70096/devopsexamapp:latest     3b0a6b09d6b9                     446MB       108MB
alpine:latest                       294b683cb724                     13MB        3.94MB
applicationdeploy-flask:latest       09229e411c84                     446MB       108MB
mysql:5.7                           4bc6bc963e6d                     700MB       149MB
sonarqube:lts-community              f709975ab31d                     1.02GB      398MB
ubuntu@ip-172-31-35-232:~$
```

Figure 39. Containers running for 2 hours and the locally available images.

```

ubuntu@ip-172-31-35-232:~$ trivy image 22bcc70096/devopsexamapp:latest
2026-09-20T09:35:37Z INFO [vuln] Vulnerability scanning is enabled
2026-09-20T09:35:37Z INFO [secret] Secret scanning is enabled
2026-09-20T09:35:37Z INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-09-20T09:35:37Z INFO [secret] Please see https://trivy.dev/docs/v0.74/guide/scanner/secret#recommendation for fast
r secret detection
2026-09-20T09:35:37Z INFO Detected OS family="debian" version="13.7"
2026-09-20T09:35:37Z INFO [debian] Detecting vulnerabilities... os version="13" pkg_num=87
2026-09-20T09:35:37Z INFO Number of language-specific files num=1
2026-09-20T09:35:37Z INFO [python-pkg] Detecting vulnerabilities...
2026-09-20T09:35:37Z WARN Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.

```

Figure 40. Trivy detects Debian 13.7 (87 packages) and the Python packages, then reports vulnerabilities.

6. The Jenkinsfile

Paste this into the pipeline job. Replace `<dockerhub-user>` with your Docker Hub username (the image must be pushed to **your** account, see Error 9). The Trivy flags are updated to the current syntax (`--scanners vuln,misconfig`), which removes the deprecation warnings seen in the console log. The *Trivy Image Scan* stage is shown in a representative form: the console log and the archived `trivy-image-report.html` confirm that such a stage ran between push and deploy.

```

pipeline {
  agent any
  environment {
    DOCKER_IMAGE = "<dockerhub-user>/devopsexamapp:latest"
    SCANNER_HOME = tool 'sonar-scanner'
  }
  stages {
    stage('Git Checkout') {
      steps {
        echo "Checking out source code..."
        git url: 'https://github.com/<your-github-user>/devops-exam-app.git',
            branch: 'master'
      }
    }
    stage('File System Scan') {
      steps {
        sh "trivy fs --scanners vuln,misconfig --format table -o trivy-fs-report.html ."
      }
    }
    stage('SonarQube Analysis') {
      steps {
        withSonarQubeEnv('sonar') {
          sh """
            ${SCANNER_HOME}/bin/sonar-scanner \
            -Dsonar.projectName=devops-exam-app \
            -Dsonar.projectKey=devops-exam-app \
            -Dsonar.sources=. \
            -Dsonar.language=py \
            -Dsonar.python.version=3 \
            -Dsonar.host.url=http://localhost:9000
          """
        }
      }
    }
    stage('Verify Docker Compose') {
      steps {
        sh 'docker compose version || { echo "Docker Compose not available"; exit 1; }'
      }
    }
    stage('Build and Push Docker Image') {
      steps {
        dir('backend') {
          script {
            withDockerRegistry(credentialsId: 'docker') {
              sh "docker build -t ${DOCKER_IMAGE} ."
              sh "docker push ${DOCKER_IMAGE}"
            }
          }
        }
      }
    }
  }
}

```

```

    }
  }
}
stage('Trivy Image Scan') {          // representative version
  steps {
    sh "trivy image --format table -o trivy-image-report.html ${DOCKER_IMAGE}"
  }
}
stage('Deploy with Docker Compose') {
  steps {
    sh '''
docker compose down --remove-orphans || true
docker compose up -d --build
timeout 120s bash -c '
  while ! docker compose exec -T mysql \
    mysqladmin ping -uroot -prootpass --silent; do
    sleep 5
  done'
sleep 10
'''
  }
}
stage('Verify Deployment') {
  steps {
    sh '''
docker compose ps -a
curl -I http://localhost:5000 || true
'''
  }
}
}
post {
  always { sh 'docker compose logs --tail=20 || true'
           archiveArtifacts artifacts: 'trivy-*.html', allowEmptyArchive: true }
  success { echo 'Deployment successful!' }
  failure { echo 'Pipeline failed. Check logs above.' }
}
}

```

NOTE: The SonarQube stage uses `-Dsonar.host.url=http://localhost:9000` because Jenkins and SonarQube run on the same EC2 host. The original course file used an older set of flags; both variants work.

7. Results and Output

7.1 Pipeline outcome

- **Build #4: SUCCESS.** The console ends with "Deployment successful!" and lists `flask_app` (Up) and `mysql_db` (Up, healthy).
- A health probe from the Docker network returned **HTTP 200** from the Flask server.
- Two artifacts were archived: **trivy-fs-report.html** (file-system scan) and **trivy-image-report.html** (image scan).
- The Docker image was pushed to Docker Hub and is present locally (446 MB on disk, 108 MB of content).

7.2 SonarQube analysis

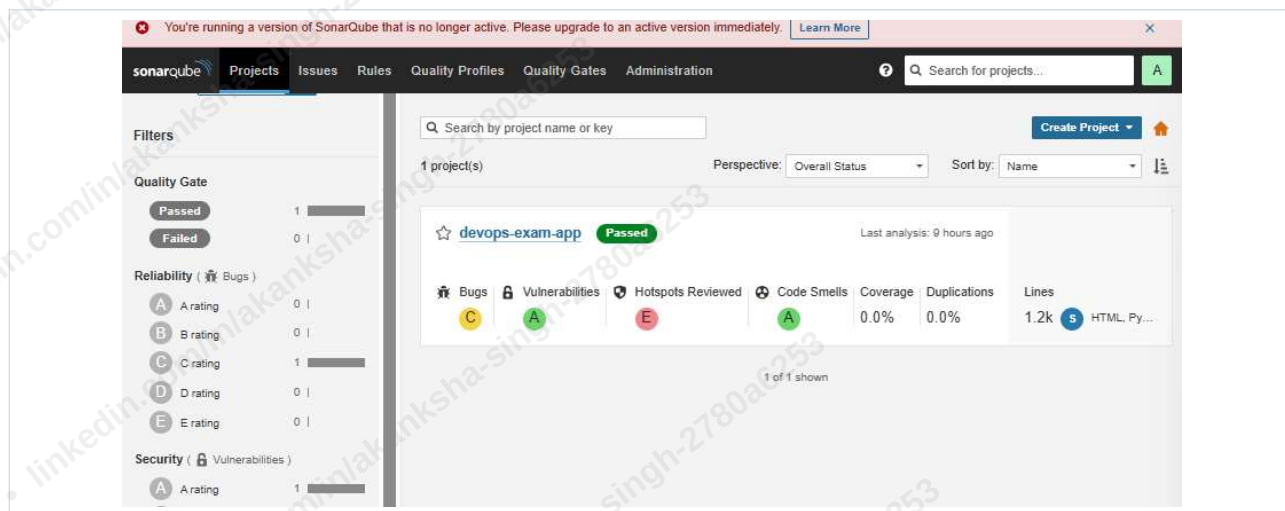


Figure 41. SonarQube project list: devops-exam-app, Quality Gate Passed.

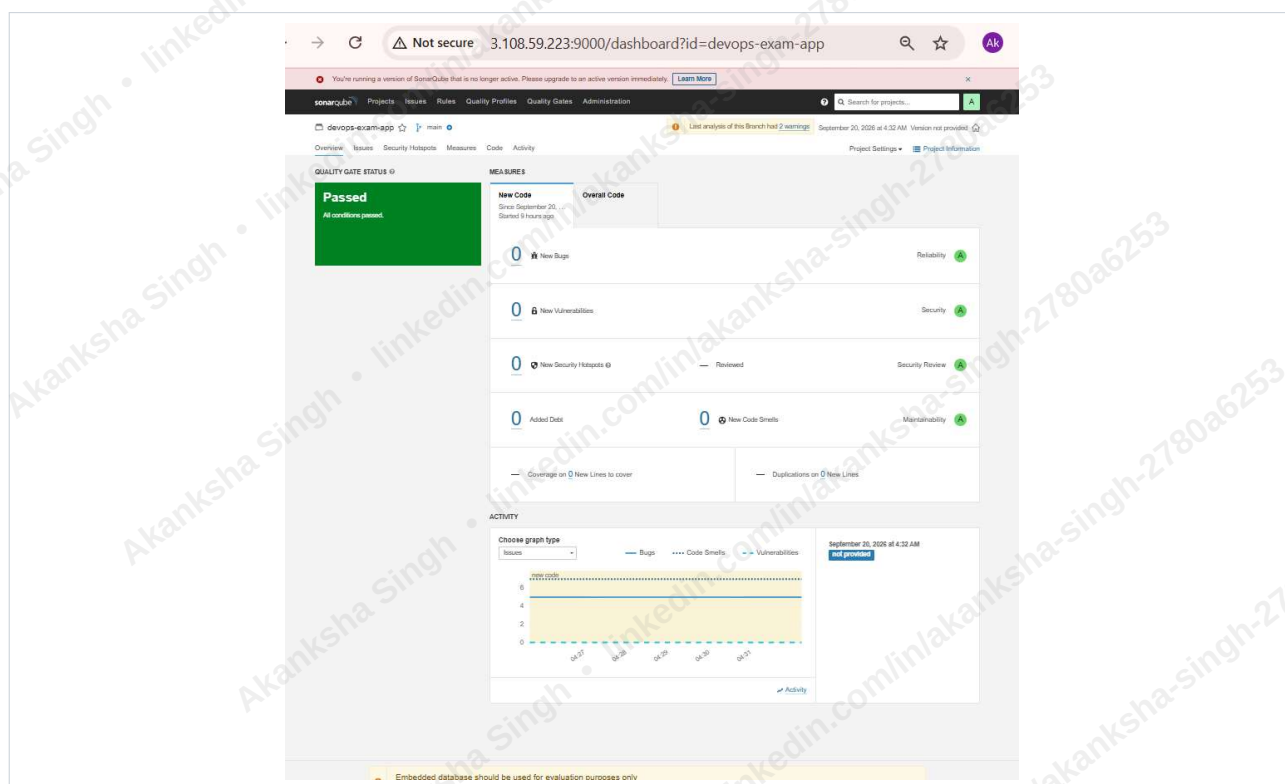


Figure 42. Dashboard, new-code view: 0 bugs, 0 vulnerabilities, 0 hotspots, 0 code smells.

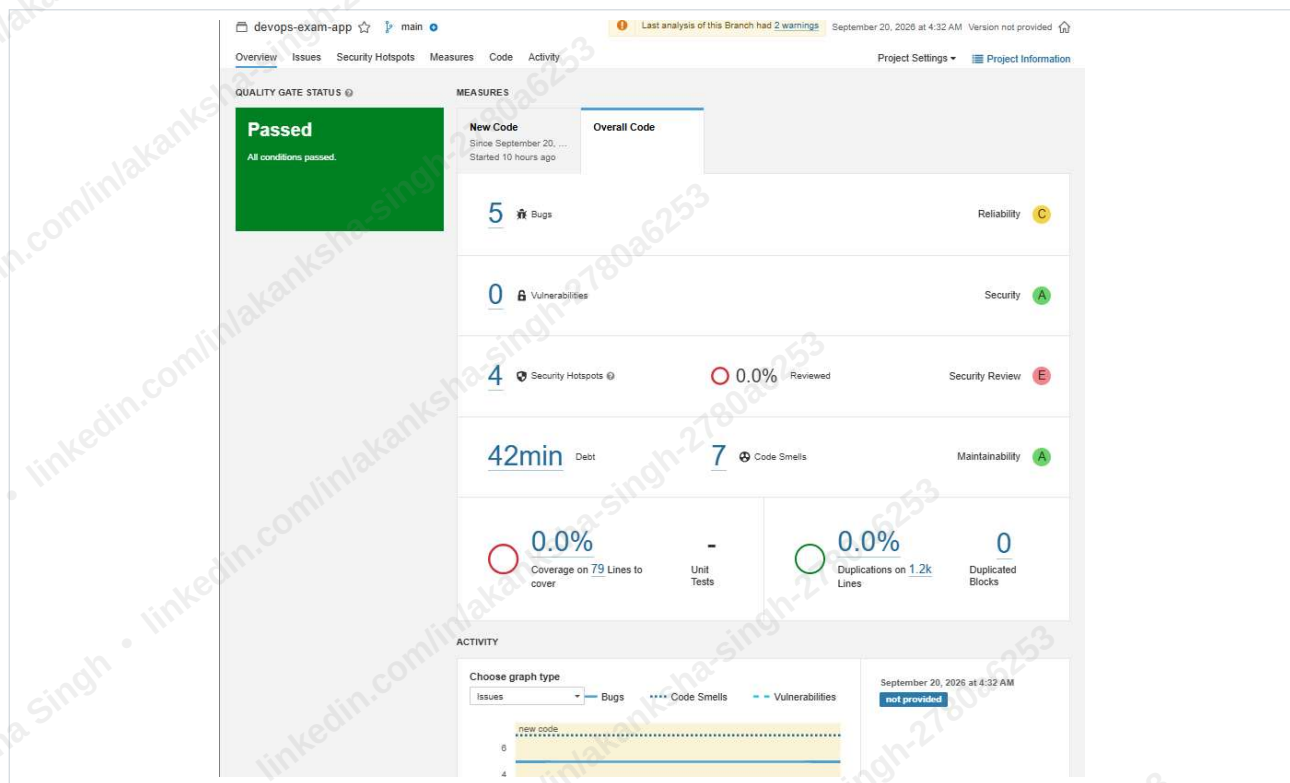


Figure 43. Overall-code view.

Metric (overall code)	Value	Rating
Bugs	5	C (reliability)
Vulnerabilities	0	A (security)
Security hotspots	4 (0% reviewed)	E (security review)
Code smells	7	A (maintainability)
Technical debt	42 minutes	-
Coverage	0.0% on 79 lines (no unit tests)	-
Duplications	0.0% on 1.2k lines	-

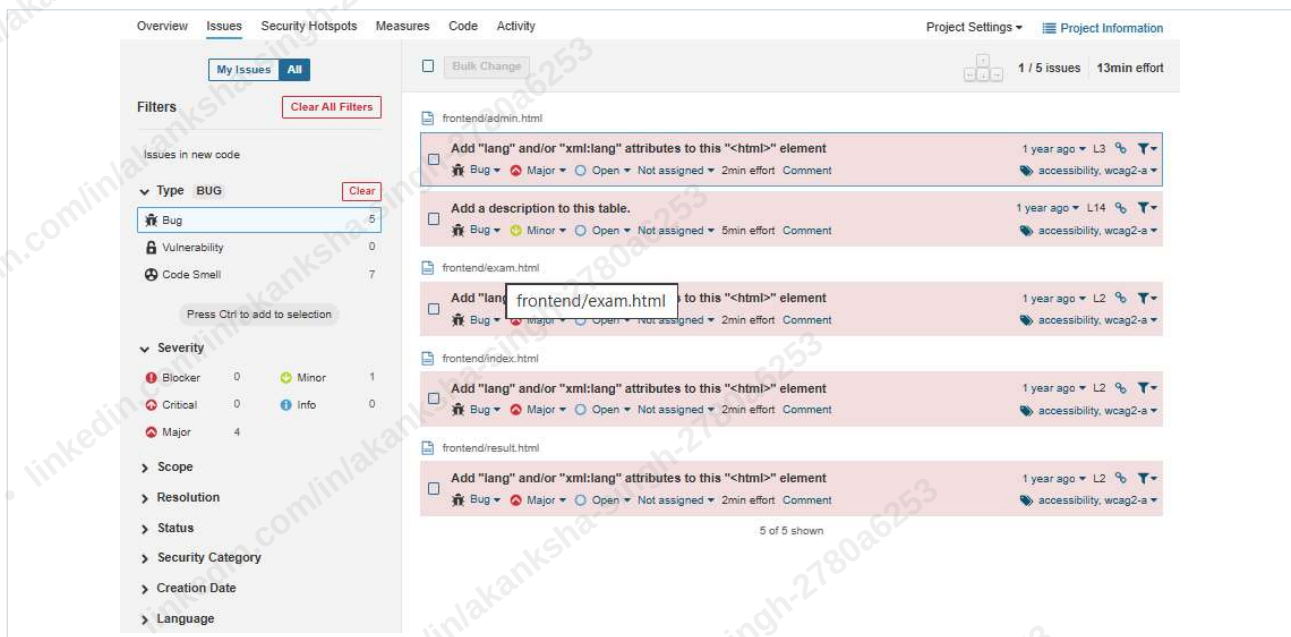


Figure 44. The five bugs: missing lang attribute on HTML pages and a table without description (accessibility, frontend files).

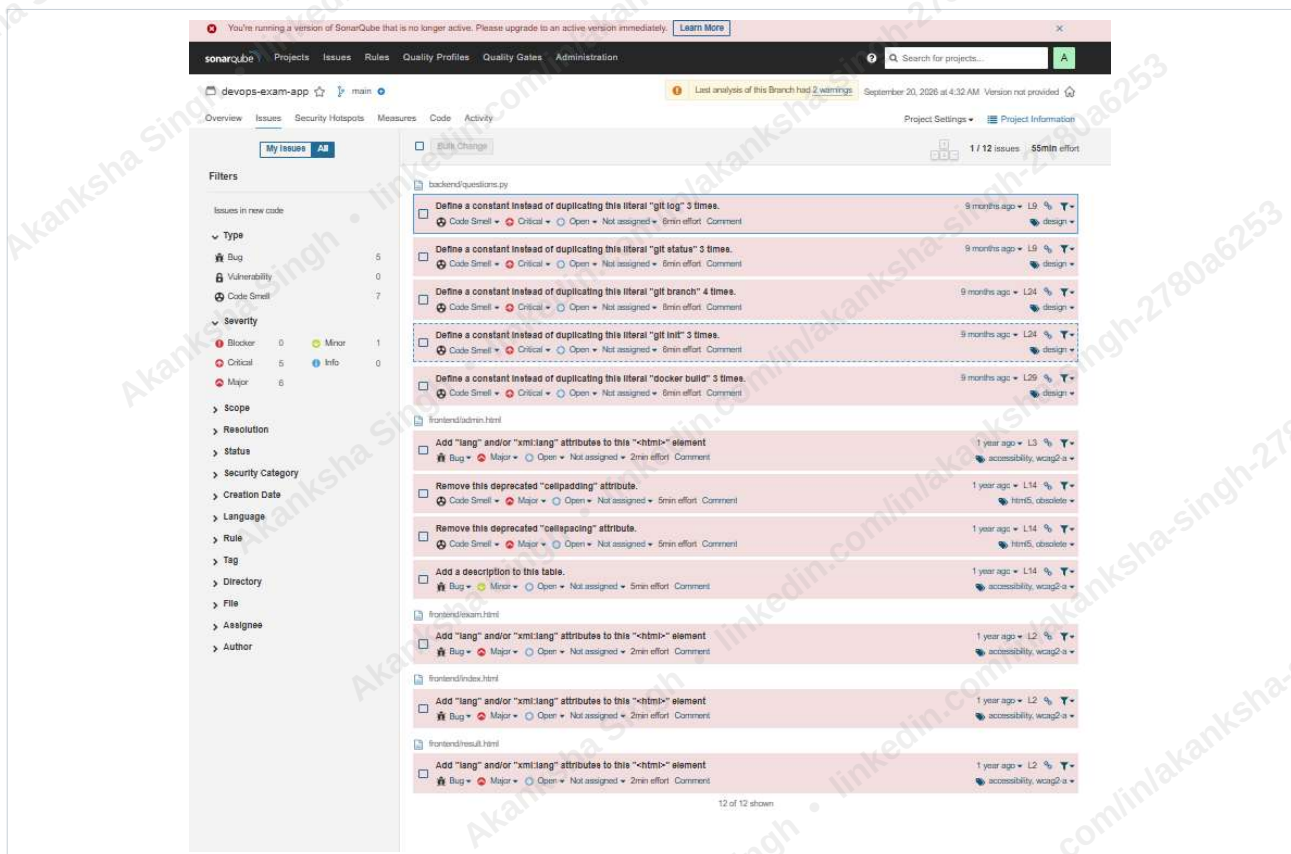


Figure 45. All 12 issues: bugs in frontend HTML plus duplicated string literals in backend/questions.py.

RESULT: The Quality Gate is **Passed** because the default gate judges *new code* since the analysis started, and that had no issues. The overall-code view still lists 12 issues, which the developer (not DevOps) is expected to fix. A gate answers one question: is this code good enough to move forward in the CI/CD pipeline?

7.3 What the application shows

The mock exam form stores each submission in MySQL. After one test attempt the table results contained one record (score 15, average 15.0000, count 1), which proves the full path browser, Flask, MySQL works.

8. Errors Faced and How They Were Solved

This is the most valuable part of the project. Each problem below actually happened during the run, with its cause and the fix that worked.

Error 1: chmod fails with "cannot access ...: No such file or directory"

Symptom	chmod: cannot access '*.sh3tierapplicationdeplyDevSecOps'
Root cause	The folder name was pasted onto the end of the command line, so the shell looked for a file called *.sh3tier....
Solution	Re-run the command on its own line.
<pre>chmod +x *.sh</pre>	

Error 2: Java 17 has no installation candidate

Symptom	Package 'openjdk-17-jre-headless' has no installation candidate.
Root cause	Ubuntu 26.04 (resolute) no longer ships Java 17 in its repositories; the script was written for older releases.
Solution	Check what exists, then install a supported version. Jenkins supports Java 17, 21 and 25.
<pre>apt-cache search openjdk grep jre-headless sudo apt-get install -y openjdk-21-jre-headless</pre>	

Error 3: Jenkins repository "is not signed" (NO_PUBKEY 7198F4B714ABFC68)

Symptom	W: OpenPGP signature verification failed ... NO_PUBKEY 7198F4B714ABFC68, then "Package jenkins has no installation candidate".
Root cause	The downloaded key jenkins.io-2023.key is outdated; the repository is now signed with a newer key.
Solution	Delete the old key and repo file, download the 2026 key, recreate the repo entry and re-run apt-get update. Verify the fingerprint if in doubt.
<pre>sudo rm -f /usr/share/keyrings/jenkins-keyring.asc /etc/apt/sources.list.d/jenkins.list sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \ https://pkg.jenkins.io/debian/jenkins.io-2026.key gpg --show-keys /usr/share/keyrings/jenkins-keyring.asc # should show ...14ABFC68</pre>	

Error 4: Jenkins page times out on port 8080

Symptom	ERR_CONNECTION_TIMED_OUT in the browser at http://<ip>:8080.
Root cause	The EC2 Security Group did not allow inbound TCP 8080. A timeout (rather than "connection refused") points to a firewall dropping packets.
Solution	Add an inbound Custom TCP rule for the port, use http:// (not https://), and confirm the public IP has not changed after a stop/start (use an Elastic IP to keep it fixed).

Error 5: docker login rejects the command

Symptom	docker: 'docker login' requires at most 1 argument.
Root cause	Typo: - u (with a space) instead of -u; the lone "-" is read as an extra argument.
Solution	Type it as one option.

```
docker login -u <dockerhub-username>
```

```
Sep 19 21:34:49 ip-172-31-35-232 jenkins[10510]: 2020-09-19 21:34:49.028700000 [id=30]
Sep 19 21:34:49 ip-172-31-35-232 systemd[1]: Started jenkins.service - Jenkins Continuous I
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker login -u 22bcc70096
Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings
Password: █
```

Figure 46. docker login prompt after the corrected command.

Error 6: Docker Scout install: "install: Permission denied"

Symptom	[error] failed to install docker-scout.
Root cause	The installer writes to /usr/local/bin, which only root can do. sudo had been applied to curl, not to the shell that runs the installer.
Solution	Put sudo on the sh side of the pipe.

```
curl -sSfL \\  
https://raw.githubusercontent.com/docker/scout-cli/main/install.sh | \\  
sudo sh -s -- -b /usr/local/bin
```

Error 7: "Directory disappeared" after the session refreshed

Symptom	cd 3tierapplicationdeployDevSecOps: No such file or directory, and the prompt changed from root@ to ubuntu@.
Root cause	The project was cloned as root, so it lives in /root. The new session was the ubuntu user whose home is /home/ubuntu.
Solution	Copy the project to the ubuntu home and take ownership, then work as ubuntu.

```
sudo cp -r /root/3tierapplicationdeployDevSecOps ~/
sudo chown -R ubuntu:ubuntu ~/3tierapplicationdeployDevSecOps
cd ~/3tierapplicationdeployDevSecOps && chmod +x *.sh
```

Error 8: Permission denied on /var/run/docker.sock

Symptom	Docker or Jenkins reports permission denied when talking to the Docker daemon.
Root cause	The user is not in the docker group. The quick workaround chmod 777 on the socket is insecure and is reset whenever Docker restarts or the server reboots.
Solution	Use group membership, then restart Jenkins and log in again.

```
sudo usermod -aG docker $USER
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
newgrp docker
```

Error 9: Jenkins build fails at Docker login: "unauthorized: incorrect username or password"

Symptom	Stage Build and Push Docker Image fails; console shows Error response from daemon ... unauthorized. The username in the log was correct.
Root cause	The password stored in the Jenkins credential was wrong (old or mistyped). Editing an existing credential keeps the old secret unless the password is explicitly replaced. A second latent problem: DOCKER_IMAGE pointed to another account (vikas4cloud/...), so a push would be denied even after login.
Solution	Test the token as the jenkins user, then delete and recreate the credential (ID docker) with the Docker Hub username and a fresh access token. Change DOCKER_IMAGE to your own username.

```

echo '<token>' | sudo -u jenkins docker login -u <dockerhub-username> --password-stdin
# Jenkins: Credentials > delete "docker" > add Username+password, ID: docker
# Jenkinsfile: DOCKER_IMAGE = "<dockerhub-username>/devopsexamapp:latest"

```

Error 10: Application on port 5000 will not open

Symptom	This site can't be reached / ERR_CONNECTION_TIMED_OUT at 3.108.59.223/5000; later, curl: Failed to connect to localhost port 5000.
Root cause	Three separate causes: (a) the URL used a slash (/5000, which means port 80 and path 5000) instead of a colon; (b) after the EC2 reboot the containers flask_app and mysql_db were Exited (255) because they had no restart policy; (c) the firewall rule for 5000 must exist in the Security Group actually attached to the instance, with source 0.0.0.0/0.
Solution	Use http://ip:5000, start the containers (MySQL first), verify locally with curl, check the Security Group, then add restart policies so it survives reboots.

```

docker ps -a
docker start mysql_db
sleep 20
docker start flask_app
sleep 10
curl -I http://localhost:5000
docker update --restart unless-stopped mysql_db flask_app sonar

```

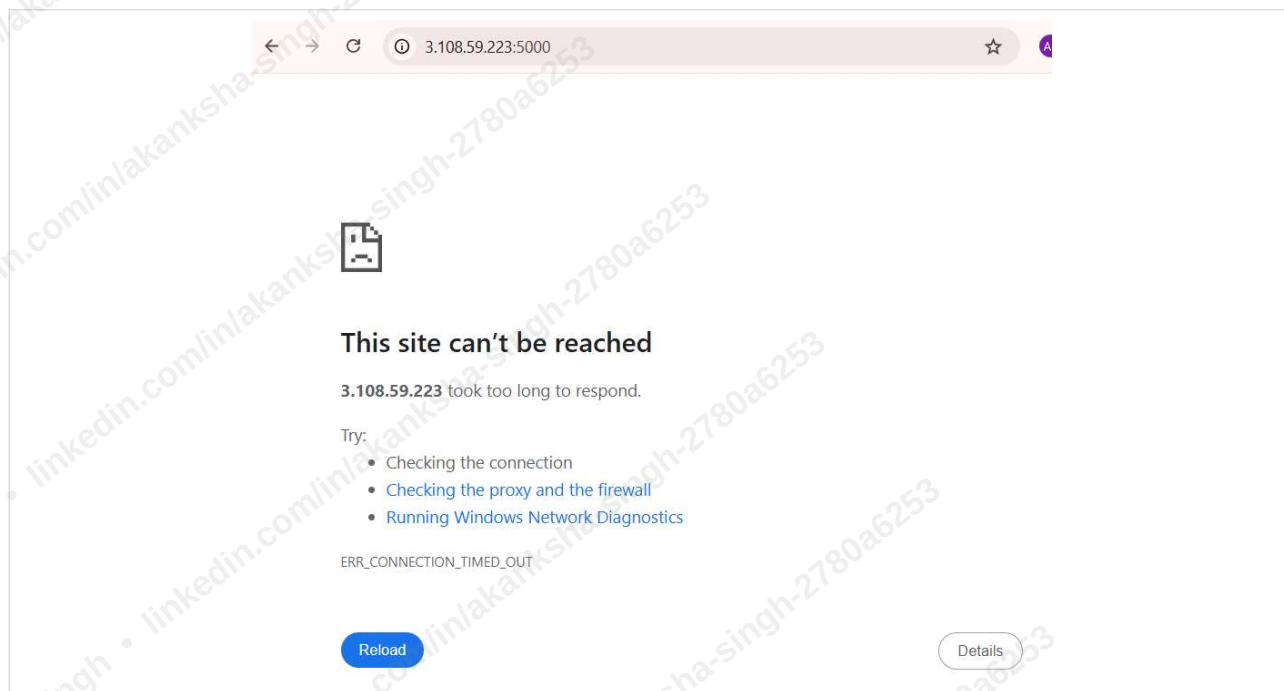


Figure 47. Browser timeout on port 5000.

```
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker ps -a
curl -I http://localhost:5000
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
0464cce93a88   applicationdeploy-flask             "python app.py"         9 hours ago    Exited (255)  5 hours ago  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
38b18c1a67f0   mysql:5.7                          "docker-entrypoint.s..." 9 hours ago    Exited (255)  5 hours ago  0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
e03a0a889e7f   sonarqube:its-community            "/opt/sonarqube/dock..." 10 hours ago    Up About a minute 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
curl: (7) Failed to connect to localhost port 5000 after 0 ms: Could not connect to server
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
e03a0a889e7f   sonarqube:its-community            "/opt/sonarqube/dock..." 10 hours ago    Up 10 minutes  0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
```

Figure 48. docker ps -a: flask_app and mysql_db Exited (255), sonar Up; curl cannot connect.

```
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$ docker start mysql_db
sleep 20
docker start flask_app
docker ps
curl -I http://localhost:5000
mysql_db
flask_app
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
0464cce93a88   applicationdeploy-flask             "python app.py"         9 hours ago    Up Less than a second 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
38b18c1a67f0   mysql:5.7                          "docker-entrypoint.s..." 9 hours ago    Up 20 seconds (healthy) 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
e03a0a889e7f   sonarqube:its-community            "/opt/sonarqube/dock..." 10 hours ago    Up 12 minutes 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
curl: (56) Recv failure: Connection reset by peer
ubuntu@ip-172-31-35-232:~/3tierapplicationdeployDevSecOps$
```

Figure 49. After starting the containers. The immediate curl gave "Connection reset by peer" (Error 11).

Error 11: curl: "Recv failure: Connection reset by peer" right after starting Flask

Symptom	curl -I http://localhost:5000 failed although docker ps showed flask_app as Up.
Root cause	The command ran a fraction of a second after the container started; Flask was not listening yet.
Solution	Wait about 10 seconds and repeat only the curl. If it keeps failing, read docker logs --tail=30 flask_app.

```
sleep 10
```

```
curl -I http://localhost:5000
```

Error 12: Trivy: "unable to find the specified image"

Symptom	MANIFEST_UNKNOWN / No such image: /devopsexampapp:latest.
Root cause	Typo in the image name (exampapp instead of examapp). Trivy also lists a harmless containerd "permission denied" line while it searches several sources.
Solution	Copy the exact name from docker images.

```
docker images
trivy image <dockerhub-user>/devopsexamapp:latest
```

Error 13: MySQL: "ERROR 1064 (42000): You have an error in your SQL syntax"

Symptom	select count (*) from results;
Root cause	A space between COUNT and the parenthesis; MySQL parses count (*) as an ordinary identifier followed by an expression.
Solution	Remove the space.

```
SELECT COUNT(*) FROM results;
```

Error 14: Deprecation warnings from Trivy and Docker Compose (warnings, not failures)

Symptom	'--security-checks' is deprecated ... '--scanners config' is deprecated; and "the attribute version is obsolete" from docker compose.
Root cause	Newer Trivy renamed the options; newer Compose ignores the top-level version key.
Solution	Use the new syntax and delete the version line from docker-compose.yml.

```
trivy fs --scanners vuln,misconfig --format table -o trivy-fs-report.html .
```

Issues still worth fixing (found while writing this report)

- **Webhook URL** was entered with a doubled scheme (http://http://...). Correct it to http://<public-ip>:8080/sonarqube-webhook/ so Jenkins can receive the Quality Gate result. Then add a waitForQualityGate abortPipeline: true step to make the gate enforce quality.
- SonarQube prints "Error when running node -v. Is Node.js available?" and the CSS sensor is skipped. Install Node.js on the Jenkins host if CSS analysis is wanted.
- SonarQube shows a banner that this LTS version is no longer active. Upgrade the image to a current LTS.
- Trivy reported "Unable to find python site-packages" (license detection skipped). Harmless for this lab.

9. Quick Verification Cheat Sheet

Check	Command
Jenkins running	<code>sudo systemctl status jenkins --no-pager</code>
Jenkins listening	<code>sudo ss -tlnp grep 8080</code>
Containers	<code>docker ps -a</code>
App health (server side)	<code>curl -I http://localhost:5000</code>
Jenkins can use Docker	<code>sudo -u jenkins docker ps</code>
Docker Hub login works	<code>docker login -u <user></code>
Container logs	<code>docker logs --tail=30 flask_app</code>
Compose logs	<code>docker compose logs --tail=50</code>
Scan image, serious findings only	<code>trivy image --severity HIGH,CRITICAL <user>/devopsexamapp:latest</code>
Make containers survive reboot	<code>docker update --restart unless-stopped mysql_db flask_app sonar</code>
Jenkins log if it will not start	<code>sudo journalctl -u jenkins -n 50 --no-pager</code>

10. What I Learned

- **Shift-left security** is practical: a single Jenkinsfile can scan files, analyse code and scan images before anything is deployed.
- **Read the error, not the tutorial.** Most failures came from a newer OS release (26.04) than the tutorial assumed: missing Java 17 and a rotated repository signing key.
- **Timeout vs refused:** `ERR_CONNECTION_TIMED_OUT` usually means a firewall or Security Group; "connection refused" means nothing is listening.
- **Linux users and homes matter:** root and ubuntu have different home directories, and Docker access is granted through the docker group, not `chmod 777`.
- **Credentials in Jenkins are opaque:** if a stored secret is wrong, recreate it. Always test the same token from the command line first.
- **Containers need restart policies;** otherwise a reboot silently takes the app down.
- **Quality Gate semantics:** the default gate evaluates new code, so a Passed gate can coexist with existing issues.
- **Order of startup:** database first, wait for healthy, then the application.
- **Tools:** Jenkins (pipelines, credentials, plugins, tool configuration), Docker and Compose, Trivy, SonarQube, Docker Scout, Docker Hub, AWS EC2 and Security Groups, basic MySQL administration.

11. Security Hardening and Next Steps

The lab optimises for speed. For a production-style setup, improve these points:

Area	Recommendation
Access control	Restrict Security Group sources to your IP or a VPN; place Jenkins and SonarQube behind HTTPS (reverse proxy such as Nginx or an ALB).

Area	Recommendation
Secrets	Keep tokens only in Jenkins credentials. Rotate any token that was ever displayed or shared. Do not keep database passwords (rootpass) in compose files; use env files or a secret manager.
Docker socket	Never chmod 777 the Docker socket. Use group membership and consider rootless or dedicated build agents.
Pipeline gates	Fail the build on serious findings: <code>trivy ... --severity HIGH,CRITICAL --exit-code 1</code> ; enforce the SonarQube gate with <code>waitForQualityGate</code> .
Extra scanners	Add Gitleaks (secrets), Checkov (IaC) and Docker Scout policy checks.
Runtime	Replace the Flask development server with a WSGI server such as Gunicorn; do not publish MySQL 3306 to the host.
Reliability	Attach an Elastic IP, add restart policies, back up MySQL data volumes.
Cost	Stop or terminate the EC2 instance when the lab is finished.

12. Interview Questions From This Project

Q. What is DevSecOps and what does shift-left mean?

Integrating security into every stage of the CI/CD pipeline and testing early (before deployment) instead of after release.

Q. Which parameter does SonarQube use to decide if code may proceed?

The Quality Gate: a set of conditions (bugs, vulnerabilities, hotspots, coverage, duplication) that returns Passed or Failed.

Q. Who fixes the bugs SonarQube reports?

The developers who own the code; DevOps wires the check into the pipeline and enforces the gate.

Q. How do you connect to a database that runs inside a container?

`docker exec -it mysql_db mysql -u root -p`, then run SQL as usual.

Q. Why did the app time out even though the container was running?

A missing Security Group rule (or wrong URL format). Timeout means the firewall dropped packets; refused means nothing listens.

Q. Why does Jenkins need to be in the docker group?

Pipeline steps call the Docker CLI as the jenkins user, which needs permission to use the Docker socket.

Q. What is the difference between trivy fs and trivy image?

fs scans project files (dependencies, IaC/config); image scans OS packages and app dependencies inside a built container image.

Q. Why use a Docker Hub access token instead of a password?

Tokens are revocable, scoped (Read & Write) and work with 2FA without exposing the account password.

13. Conclusion

The project delivers a complete, working DevSecOps pipeline on a single EC2 instance: source checkout, Trivy scanning, SonarQube analysis with a Quality Gate, Docker image build and push, and automated deployment of a Flask and MySQL application with Docker Compose. Along the way, fourteen real-world problems were diagnosed and solved, which turned a copy-and-paste tutorial into practical troubleshooting experience. The same pattern scales to larger applications by adding more scanners, stricter gates and production-grade infrastructure.

Credits and references

- Course template and installation scripts: CloudDevOpsHub, repository [CloudDevOpsHub/3tierapplicationdeployDevSecOps](#) on GitHub.
- Tools: Jenkins ([jenkins.io](#)), Trivy ([trivy.dev](#)), SonarQube ([sonarsource.com](#)), Docker ([docs.docker.com](#)).
- Screenshots in this report are from my own run of the project; secrets and personal email addresses have been removed.